Università degli Studi di Genova

Course: High Performance Computing

Performance Optimization and Scaling of Large-Scale Matrix Multiplication

Luca Barbati, 5082540

Wassim Fatnassi, 4684857

Roberto Lazzarini, 4937188

Contents

1 Introduction **3**
1.1 Problem Definition & Objectives . . . 3
1.2 Experimental Setup . . . 3
1.2.1 Architecture and Compute Capabilities . . . 4
1.2.2 Cache Hierarchy and Vectorization . . . 4
1.3 Metrics and Profiling Tools . . . 4
1.3.1 Roofline Model in Detail . . . 5
1.4 Pipeline . . . 5

2 Sequential Implementation **7**
2.1 Hotspot Identification . . . 7
2.2 Implementations . . . 8
2.2.1 Baseline: Row-Major and Spatial Locality . . . 8
2.2.2 Optimized Version: Loop Tiling technique . . . 8
2.2.3 Library Reference: CBLAS . . . 11
2.3 Compiler Analysis . . . 11
2.4 Additional Code Optimizations . . . 12
2.4.1 Register Promotion: the `temp_a` variable . . . 12
2.4.2 Minimum management . . . 14
2.5 Results & Comparison . . . 15
2.5.1 Roofline Model . . . 15

3 Shared Memory: OpenMP **17**
3.1 Implementations . . . 17
3.1.1 Parallelized Baseline . . . 17
3.1.2 Optimized Version . . . 18
3.1.3 Library Reference: CBLAS . . . 18
3.2 Additional Code Optimizations . . . 19
3.2.1 Restrict keyword . . . 19
3.2.2 OpenMP Scheduling Strategies . . . 19
3.3 Ablation Study . . . 19
3.4 Results & Comparison . . . 21
3.4.1 Roofline Model . . . 21

4 Distributed Computing: MPI **25**
4.1 Implementations . . . 25
4.1.1 Parallelized Baseline . . . 25
4.1.2 Optimized Version . . . 26
4.1.3 Library reference: ScaLAPACK . . . 28
4.2 Ablation study . . . 29
4.2.1 Oversubscription is beneficial for the optimized version . . . 29
4.2.2 Oversubscription harms the library's performance . . . 30
4.3 Results & Comparison . . . 31

5 Heterogeneous Computing: CUDA **33**
5.1 Implementations . . . 33
5.1.1 Preamble: the shift to Single Precision . . . 33
5.1.2 Parallelized Baseline . . . 34
5.1.3 Optimized Version . . . 35
5.1.4 Library reference: cuBLAS . . . 37
5.2 Additional Code Optimizations . . . 37
5.2.1 Data type float4 . . . 37
5.3 Hyperparameter Tuning . . . 38
5.4 Results & Comparison . . . 38

6 Conclusions & Results **41**
6.1 Sequential . . . 41
6.1.1 Experimental Data and Speedup . . . 41
6.1.2 Throughput and Roofline Metrics . . . 41
6.2 OpenMP . . . 41
6.2.1 Execution Times and Scheduling Policies . . . 41
6.2.2 Parallel Metrics (Speedup and Efficiency) . . . 42
6.2.3 Parallel Throughput and Roofline Metrics . . . 42
6.3 MPI . . . 42
6.3.1 Experimental Data . . . 42
6.3.2 Distributed Speedup, Efficiency, and Throughput . . . 42
6.4 CUDA . . . 43
6.4.1 Theoretical Peak and Roofline Parameters . . . 43
6.4.2 Experimental Results and Throughput . . . 43
6.4.3 Experimental Results and Throughput . . . 43
6.5 Final Comparison . . . 44
6.5.1 Global Performance Comparison ($N = 10000$) . . . 44
6.5.2 Conclusions . . . 44

1 Introduction

1.1 Problem Definition & Objectives

The objective of this project is the optimization and parallelization of the multiplication of two square matrices of size $n \times n$, defined as:

$$C_{i,j} = \sum_{k=0}^{n-1} A_{i,k} \cdot B_{k,j}$$

Despite its simple mathematical definition, the algorithm presents a computational complexity of $O(n^3)$. In modern computing architectures, however, the primary bottleneck is rarely the processor's speed, but rather the Memory Wall, the significant performance gap between fast CPU cores and relatively slow main memory. Consequently, the efficient utilization of the cache hierarchy plays a fundamental role in achieving high performance. The primary objective of the project is the exploration of the matrix multiplication problem and subsequently the transformation of the baseline sequential algorithm into a suite of high-performance implementations. Our study and analysis span from optimizations of the sequential code to comprehensive parallelization strategies across distinct programming paradigms. Specifically, we leverage and analyze shared-memory (OpenMP), distributed-memory (MPI), and GPU acceleration (CUDA) architectures.

1.2 Experimental Setup

To contextualize performance results, it is essential to define and specify the hardware environment. The performance of an algorithm depends heavily on how well the code interacts with the specific hardware architecture and memory organization of the underlying system. The main specifications are summarized in Table 1.

Component	**Specification**
System	HP Z2 Tower G9 Workstation
Processor	Intel Core i9-12900K (12th Gen, Alder Lake)
Architecture	Hybrid: 8 P-Cores (16 threads) + 8 E-Cores (8 threads)
Logical Threads	24
Clock Frequency	800 MHz (Base) / 4.02 GHz (Max OS Reported)
L1 Cache	640 KiB (Data) / 768 KiB (Instruction)
L2 Cache	14 MiB (Cumulative across cores)
L3 Cache	30 MiB (Shared Last Level Cache)
Main Memory	62 GiB RAM
Storage	953.9 GiB NVMe SSD
Vector Capabilities	AVX2, FMA (SIMD)
Operating System	Debian GNU/Linux 12 (Kernel 5.18.0)

Table 1: Hardware and Software specifications of the system.

1.2.1 Architecture and Compute Capabilities

The benchmarks are executed on an HP Z2 Tower powered by an Intel Core i9-12900K processor. As detailed in Table 1, the system provides 24 logical threads, which constitute the main execution environment for our parallelization phase.
The availability of 62 GiB of main memory allows the safe allocation of large-scale matrices, by avoiding disk swapping (I/O latency) and ensuring that all measurements strictly reflect pure computational and memory performance.

1.2.2 Cache Hierarchy and Vectorization

The hierarchy memory represents the most critical factor in overcoming the Memory Wall. The optimization strategies implemented in this project are tailored around the processor's multi-level cache structure, trying to leverage the following aspects:

- **Cache Locality:** the L1 data cache is the primary target for our optimizations, while the larger L2 and L3 caches act as secondary staging areas to hold data, maximizing its reuse before it is overwritten.
- **Data-Level Parallelism:** the processor supports AVX2 and FMA (Fused Multiply-Add) instruction sets. These capabilities allow the execution of multiple operations in a single clock cycle via SIMD (Single Instruction, Multiple Data) execution.

1.3 Metrics and Profiling Tools

To ensure a rigorous analysis, the performance of each implementation is assessed using the following metrics:

- **Execution Time:** the total time required to complete the computation, expressed in seconds. To ensure statistical reliability and mitigate system noise, each reported value is the average of multiple independent runs under identical conditions.
- **Throughput:** represents the number of floating-point operations executed per second. For an $n \times n$ matrix multiplication, it is calculated as:

  $$\text{GFLOPS} = \frac{2 \cdot n^3}{T \times 10^9}$$

  It serves as a fundamental indicator to compare performance across different problem sizes (n) and hardware architectures.

For the parallel implementations, the following specific metrics are utilized:

- **Speedup:** the ratio between the best sequential execution time (T_{seq}) and the parallel execution time (T_{par}) for a given number of processing units (p).

  $$S(p) = \frac{T_{seq}}{T_{par}(p)}$$

- **Efficiency:** a measure of how effectively the computational resources are utilized, reflecting the overhead of parallelization (e.g., communication, synchronization).

  $$E(p) = \frac{S(p)}{p}$$

The main tools used to support the development flow and conduct a performance evaluation are:

- **Compilers and Optimization Flags:** compilers are the main tool of High Performance Computing. To maximize hardware utilization and extract peak performance, aggressive optimization flags were systematically applied. For the sequential and OpenMP implementations, we selected the optimal compiler between the GNU Compiler Collection (`gcc`) and the Intel oneAPI Compilers (`icc/icx`) through the ablation study detailed in Section 2.3. Instead, the MPI and GPU sections were compiled using respectively the Intel MPI compiler wrapper (`mpiicx`) and the NVIDIA CUDA compiler (`nvcc`).

- **Roofline Model:** this framework is used to evaluate the quality of our optimizations, allowing to visually relate measured performance (GFLOPS) to the machine's peak computational limits and memory bandwidth. By calculating the Arithmetic Intensity (FLOPs per byte), the model identifies whether an implementation is *memory-bound* (limited by data transfer) or *compute-bound* (limited by CPU power).

1.3.1 Roofline Model in Detail

The Roofline Model identifies the efficiency of our matrix multiplication implementation by characterizing two distinct hardware boundaries:

- **Performance (Y-axis):** measured in GFLOPS (Giga Floating-point Operations Per Second), it represents the absolute computing throughput.

- **Arithmetic Intensity (X-axis):** measured in FLOP/Byte, it expresses the ratio of total floating-point operations to the total data traffic (bytes) transferred between the CPU and the memory hierarchy.

By plotting our implementations, we are able to identify two hardware constraints:

- **Computational Peaks (horizontal lines):** these represent the maximum throughput the CPU can achieve. Kernels near these lines are "Compute Bound" and require better vectorization or instruction-level parallelism.

- **Memory Bandwidth (oblique lines):** these slopes represent the maximum data transfer rates of the memory hierarchy (L1, L2, L3, and DRAM). Kernels along these slopes are "Memory Bound," meaning the CPU is stalled waiting for data.

The intersection of these boundaries is the Ridge Point, which defines the minimum Arithmetic Intensity needed to potentially reach peak CPU performance. In our analysis, conducted via Intel Advisor, each marker on the plot represents a specific loop or function; the size of each marker is proportional to its execution time, helping us to identify "hotspots". The color coding summarized in Table 2 provides an immediate diagnostic:

1.4 Pipeline

The development process of the project is structured as follows:

- **Hotspot Identification**: localization of the code sections that represent the computational bottleneck.

Color	Meaning	Status
Red	Critical hotspot: consumes significant computational time.	High priority optimization.
Yellow	Medium hotspot: moderate impact on the execution.	Secondary optimization target.
Green	Efficient performance: the loop is well-optimized or has a negligible impact.	No further action required.

Table 2: Classification based on Intel Advisor's color-coding scheme.

- **Sequential Optimization**: determination of the optimal sequential technique, leveraging advanced compiler optimization flags.
- **Compiler Performance Comparison**: comparison of the performance obtained using different compilers, determining the best compiler and flag sequence, which will be used in the following parts which support them.
- **Multi-Paradigm Parallelization**: implementation of specialized versions for different parallel programming paradigms:
  - **OpenMP**: shared-memory parallelism, focusing on optimal thread scheduling and workload balancing across CPU cores.
  - **MPI**: distributed-memory computing, implementing data decomposition strategies to minimize inter-node communication overhead.
  - **CUDA**: exploiting the massive parallelism of GPU through efficient kernel design and memory hierarchy management.
- **Comparative Analysis**: a comprehensive evaluation and cross-comparison of all developed implementations. This analysis aims to clear how different parallel programming paradigms perform, as well as how they scale compared to a fine-tuned sequential baseline and industry-standard libraries.

Note: to ensure statistical reliability and mitigate operating system noise, each reported execution time in the project represents the mean of three independent runs.

2 Sequential Implementation

In this section, we present and analyze the progression of sequential optimizations applied to the baseline matrix multiplication. While the theoretical computational complexity remains fixed at $O(n^3)$, the primary objective is to bridge the performance gap caused mainly by the Memory Wall, leveraging the CPU's cache hierarchy.

2.1 Hotspot Identification

Before applying any structural transformations, the first step is to identify the computational bottleneck. According to Amdahl's Law, optimizations should be strictly related to the sections of the code that consume the majority of the execution time (*hotspots*). To isolate these critical sections, we tested the baseline version of the matrix multiplication compiled with GCC and no optimization flags, separating the execution time of the pure mathematical computation (nested loop) from the overhead of memory allocation, matrix initialization, and I/O operations. As demonstrated in Table 3, the nested loop essentially consists on the entire runtime of the program; for a matrix of size $n = 5000$, it consumes 99.88% of the total execution time.

Matrix Size (n)	Total Time (s)	Critical Section (s)	Overhead (s)	Hotspot
2500	26.72	26.55	0.16	99.38%
5000	212.48	212.23	0.24	99.88%

Table 3: Hotspot identification: the critical section (nested loops) defines the overall execution time. The results were obtained using the GGC compiler without any optimization flags.

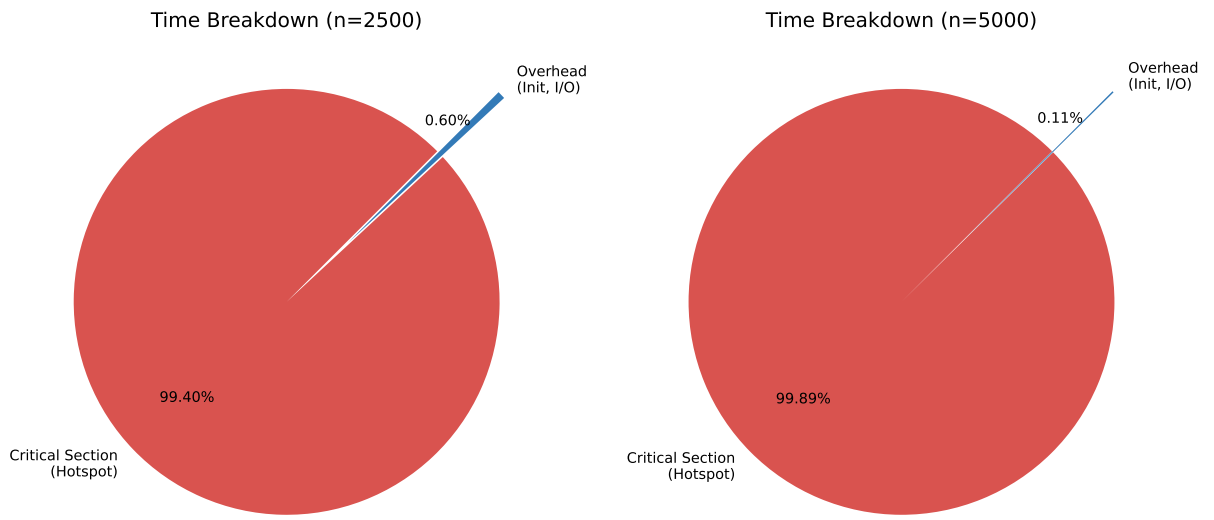

Figure 1: Visualization of the execution time distribution for $n = 5000$, highlighting the absolute dominance of the hotspot.

2.2 Implementations

2.2.1 Baseline: Row-Major and Spatial Locality

The initial baseline, implemented in `matrixmult.c`, introduces a first optimization: reordering of the nested loops from the standard $i - j - k$ to $i - k - j$.

```
// Matrix multiplication through i-k-j order.
for (i = 0; i < n; ++i)
    for (k = 0; k < n; k++)
        for (j = 0; j < n; ++j)
            c[i][j] += a[i][k] * b[k][j];
```

Listing 1: Loop reordering for spatial locality in matrixmult.c

This structural change is specifically designed to align with the Row-Major memory storage convention of the C language. By varying the j index in the innermost loop, the algorithm accesses the elements of matrices B and C in a contiguous and linear manner. In this way, for the entire duration of the inner loop, the value `a[i][k]` remains constant, allowing the compiler to load it once into a CPU register and reuse it n times. Since our architecture utilizes a 64-byte cache line, each memory fetch brings 8 double-precision elements (each 8 bytes) from RAM to the L1 cache. The Row-Major pattern ensures that all 8 elements are processed before the cache line is replaced, maximizing Spatial Locality and, consequently, reducing the overhead caused by jumping across non-contiguous memory addresses.

2.2.2 Optimized Version: Loop Tiling technique

As the matrix size n scales beyond the cache capacity, the linear access pattern fails because data is replaced from the cache before it can be reused for the next iteration. To mitigate this bottleneck, the optimized version implemented in `matrixmult_opt.c` applies the Loop Tiling technique.

```
// External loops: navigation through the Tiles using a step of B
for (ii = 0; ii < n; ii += B) {
        i_limit = min(ii + B, n);
        for (kk = 0; kk < n; kk += B) {
            k_limit = min(kk + B, n);
            for (jj = 0; jj < n; jj += B) {
                j_limit = min(jj + B, n);
                // Internal loops: computation within single Tile
                for (i = ii; i < i_limit; i++) {
                    for (k = kk; k < k_limit; k++) {
                        for (j = jj; j < j_limit; j++) {
                            c[i][j] += a[i][k] * b[k][j];
                        }
                    }
                }
            }
        }
    }
```

Listing 2: Six-loop Tiling structure in matrixmult_opt.c

Loop Tiling Technique
The Loop Tiling technique is a structural transformation which can only be applied to perfectly nested loops that are free from loop-carried dependencies, conditions fully satisfied by the standard matrix multiplication algorithm.
This technique consists of partitioning the $n \times n$ matrices into smaller $B \times B$ sub-matrices, called **tiles**. To achieve this, the traditional three-loop architecture is transformed into a six-loop structure, where the three outermost loops (ii, jj, kk) need to navigate between the blocks advancing by step of size B, while the inner loops (i, j, k) perform the multiplication within the boundaries of the current tile, a small portion of data which would fit in the L1 or L2 cache.
In the baseline implementation, a large n causes the rows of matrix B to be replaced from the cache before the k loop can reuse them, while by restricting the innermost loops to a maximum length of B, the active working set fits within the cache. Consequently, when the processor requests the element `b[k][j]`, it is highly probable that it is already in cache from the previous iteration. This drastically maximizes Temporal Locality and eliminates slow fetches from the main RAM.
A critical detail is handling matrix boundaries when the dimension n is not an exact multiple of the tile size B. In this case, to prevent the Segmentation Faults caused by the excess of the matrix boundaries, the inner loops use a `min()` function to dynamically set their upper bounds, safely truncating the block without accessing invalid memory.

Tile size: parameter B
The selection of the optimal block size B is critical to ensure that the $B \times B$ sub-matrices involved in the inner computation fit entirely within the fast caches, maximizing the benefits of Loop Tiling.
If B is too large the tiles will exceed the cache capacity, causing the data to be evicted before it can be fully reused; contrarily, if B is too small the algorithm introduces excessive overhead due to the many nested loops and fails to fully exploit the available cache space.
To determine the optimal value for B, which should be a multiple of the cache line size for efficient memory access, an empirical ablation study was conducted. The resulting execution times, reported in the following Table 4 and in the Figure 2, demonstrate that the optimal tile size is $B = 64$.

Tile Size (B)	**Execution Time (s)**
8	42.65
16	27.80
32	27.21
64	**22.53**
128	32.77
256	31.23

Table 4: Ablation study for the tile size B, tested with matrix size $n = 8000$. The results were obtained using the ICX compiler with the flag configuration: `-O3 -xHost`.

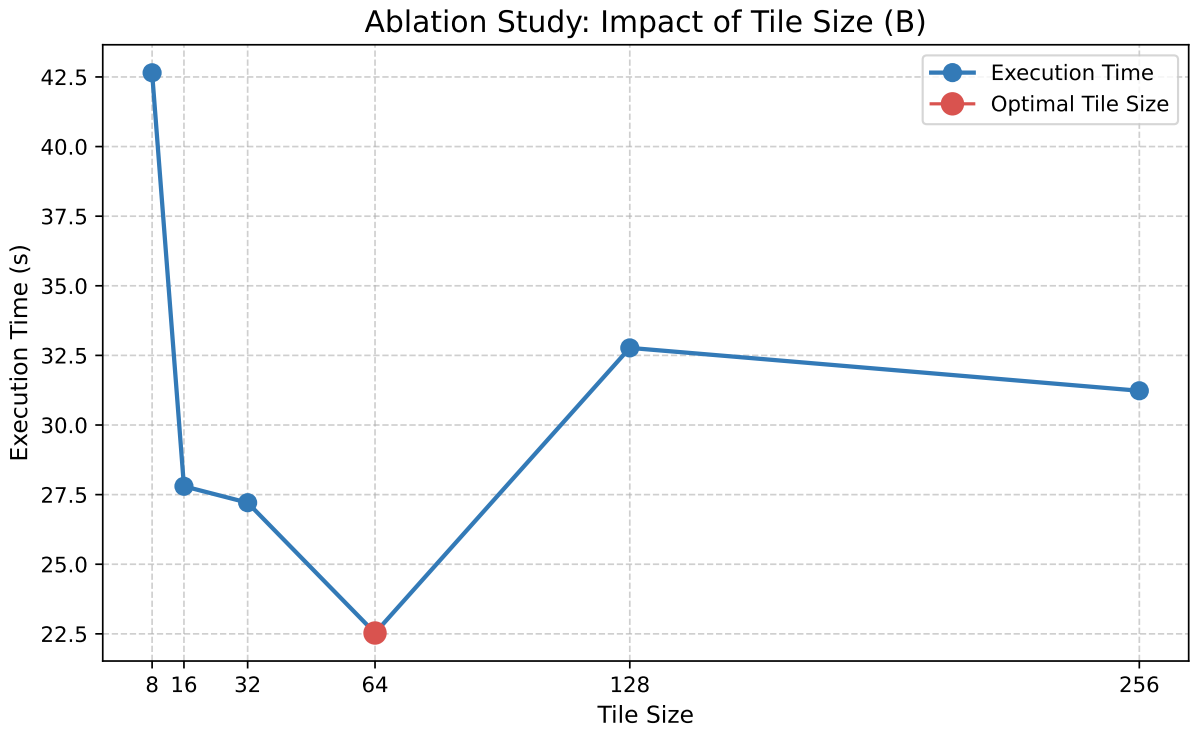

Figure 2: Impact of the tile size (B) on the execution times.

Memory Alignment: `aligned_alloc`
To further optimize the computation and fully exploit the processor's vectorization capabilities, the `matrixmult_opt.c` version replaces the standard `malloc` with `aligned_alloc`.

```
// 64 bytes to match Cache Line size and SIMD vectorization
size_t alignment = 64;
size_t matrix_size = sizeof(double[n][n]);

// Force matrix_size to be a multiple of the alignment
if (matrix_size % alignment != 0) {
    matrix_size = ((matrix_size / alignment)+1) * alignment;
}

// Memory allocation
double (*a)[n] = aligned_alloc(alignment, matrix_size);
double (*b)[n] = aligned_alloc(alignment, matrix_size);
double (*c)[n] = aligned_alloc(alignment, matrix_size);
```

Listing 3: Alignment implementation in matrixmult_opt.c

Modern CPUs utilize AVX2 instructions to process 256-bit vectors in a single clock cycle. If a matrix is allocated at an unaligned memory address, the CPU might need to fetch two separate cache lines to read a single vector.
By using `aligned_alloc`, we ensure that the starting memory address of every matrix is strictly aligned to a 64-byte boundary (the exact size of an L1 cache line). This guaranties optimal and contiguous data loading for SIMD operations, obtaining a substantial increase in performance, as demonstrated in Table 5.

Matrix Size (n)	malloc (s)	aligned_alloc (s)
8000	37.11	22.53
10000	71.45	43.91
15000	235.02	178.37

Table 5: Execution time comparison demonstrating the impact of memory alignment via `aligned_alloc`. The results were obtained using the Intel ICX compiler with the flag configuration: `-O3 -xHost`.

2.2.3 Library Reference: CBLAS

To evaluate the performance of our implementations, we developed `matrixmult_library.c`, which utilizes the highly optimized `cblas_dgemm` routine provided by professional BLAS (Basic Linear Algebra Subprograms) library, accessed by including the `#include <cblas.h>` and `#include <openblas_config.h>` headers.

```
// C = (alpha*A*B) + (beta*C)
// - CblasRowMajor: aligns with C-style 2D arrays memory layout
// - CblasNoTrans: matrices are passed as-is (no transposition)
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
            n, n, n,
            alpha,
            &a[0][0], n,
            &b[0][0], n,
            beta,
            &c[0][0], n);
```

Listing 4: CBLAS routine call.

Although our version `matrixmult_opt.c` successfully implements Single-Level Tiling and locality principles, professional BLAS libraries achieve near-theoretical peak performance by employing advanced low-level techniques. The main differences are:

- **Multi-level Tiling:** unlike our single-block approach, CBLAS hierarchically partitions the matrices to fit perfectly into all the different cache levels.
- **Data Packing:** the library temporarily copies and rearranges the matrix blocks in memory before computing, forcing the data to be perfectly contiguous to avoid memory access delays.
- **Assembly Micro-kernels:** the mathematical operations are not written in standard C, but in low-level Assembly language. This keeps the active data strictly within the CPU's vector registers, minimizing even the fast L1 cache accesses.

By adopting these sophisticated techniques and mechanisms, the library implementation achieves near-theoretical peak performance by the workstation.

2.3 Compiler Analysis

Table 6 details the compiler optimization flags evaluated to determine the optimal configuration for execution performance.

Compiler Flags	Description
`-O1`	Enables basic optimizations aimed to reduce execution time without significantly increasing compilation time.
`-O2`	Enables a larger and more aggressive set of optimizations.
`-O3`	Enables the most aggressive, high-level optimizations, explicitly including loop unrolling and auto-vectorization (SIMD).
`-Ofast` \ `-fast`	Disables strict IEEE math compliance to maximize floating-point throughput, building upon the `-O3` optimizations.
`-march=native` \ `-xHost`	Optimizes code specifically for the host architecture, unlocking advanced instruction sets like AVX2 and FMA.
`-lm`	Links the standard C mathematical library.

Table 6: Summary of compiler optimization flags evaluated during the Ablation Study.

Note: the nomenclature of specific flags for GNU (GCC) and for Intel compilers (ICC/ICX) are different and the correspondences are the following:

- GCC: -march=native, -Ofast
- ICC \ ICX: -xHost, -fast

Compiler Flags	GCC	ICC	ICX
No flags (Baseline)	978.69	75.85	97.86
`-O1`	164.18	167.77	253.70
`-O2`	185.97	68.39	82.29
`-O3`	60.09	68.55	170.72
`-O3 -march=native`\`-xHost`	30.20	44.03	**22.55**
`-Ofast`\`-fast`	30.30	44.50	22.98

Table 7: Execution times ($n = 8000$) across different compilers and optimization levels.

Table 7 and Figure 3 summarize the performance obtained by the ablation study on the GNU Compiler Collection (`gcc`) and Intel compilers (`icc`,`icx`). As evidenced by the results, the Intel ICX compiler with the `-O3` and `-xHost` optimization flags achieves the highest performance. Consequently, this specific configuration will be adopted as the definitive for all subsequent tests.

2.4 Additional Code Optimizations

2.4.1 Register Promotion: the `temp_a` variable

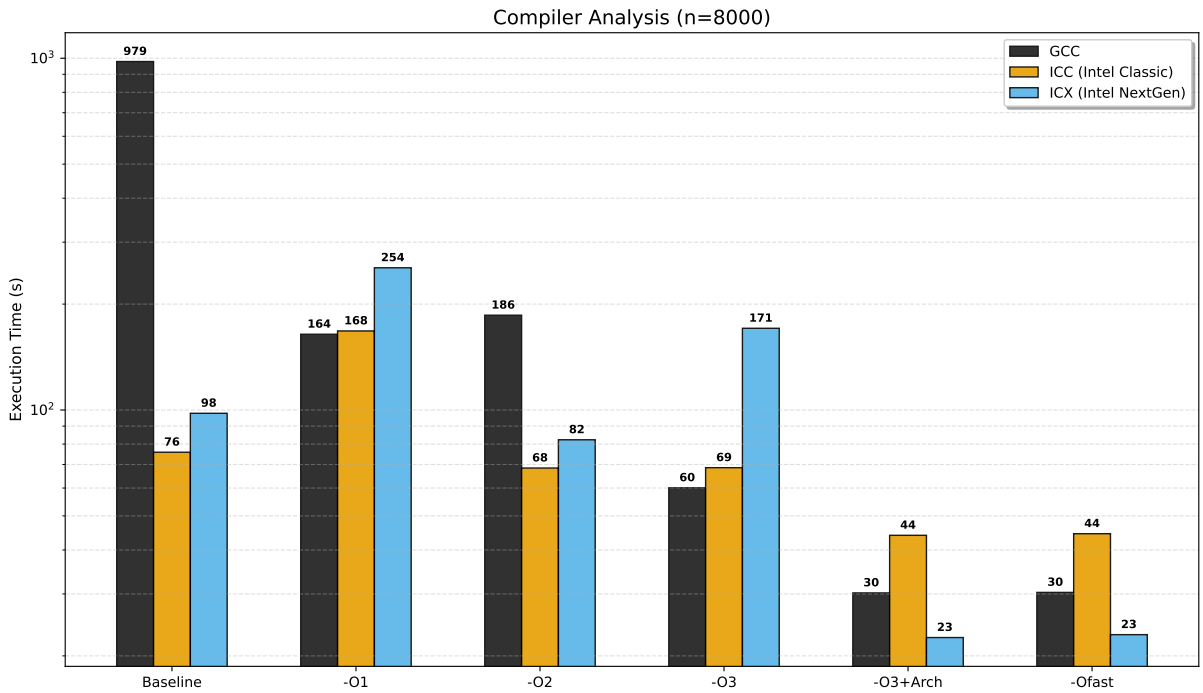

Figure 3: Performance evaluation of compilers across various optimization levels. The logarithmic scale emphasizes the dramatic reduction in execution time achieved by enabling architecture-specific flags.

```
// Internal loops: computation within the single Tile
for (i = ii; i < i_limit; i++) {
    for (k = kk; k < k_limit; k++) {
        // Register Promotion technique
        temp_a = a[i][k];
        for (j = jj; j < j_limit; j++) {
            c[i][j] += temp_a * b[k][j];
        }
    }
}
```

Listing 5: Register Promotion technique

Trying to further increase performance, we introduced a local scalar variable `temp_a` to store the operand `a[i][k]` before entering the innermost j-loop, in order to keep this constant value in a physical fast CPU register. Through this manual optimization known as Register Promotion, by explicitly extracting the value into `temp_a`, we force the compiler to "lock" this constant value into a physical CPU register with no latency, eliminating redundant cache reads per iteration.
Without adopting this technique, the compiler might conservatively reload `a[i][k]` from the cache at every single iteration of j. This happens because compilers must account for potential *pointer aliasing*, the risk that writing to matrix C might inadvertently modify the memory of matrix A. Our tests in Table 8 confirms the effectiveness of this technique but also highlights the intelligence of modern compilers: using the `gcc` compiler without any optimization flags, Register Promotion yields a significant performance improvement; instead, using advanced compilers or optimization flags (like `-O3`), the execution times

become identical. This demonstrate that compilers can automatically handle this scenario detecting as indices i and k do not vary within the j-loop and autonomously "promoting" the memory access to a CPU register, making our manual implementation redundant.

Matrix Size (n)	Compiler	Baseline (s)	Reg. Promotion (s)
2500	GCC	250.29	209.17
2500	ICX	10.15	10.16
8000	ICX	41.78	41.81

Table 8: Performance impact of manual Register Promotion across the different compilers without any optimizations flags.

2.4.2 Minimum management

To dynamically calculate the bounds for tiles to avoid invalid memory accesses, the algorithm repeatedly evaluates the minimum between the current tile limit and the matrix dimension n. Instead of relying on a standard library function, we implemented a custom preprocessor macro:

```
#define min(a,b) (((a) < (b)) ? (a) : (b))
```

Listing 6: Custom Preprocessor Macro for minimum calculation

While a standard function call introduces unnecessary architectural overhead that compounds significantly when executed thousands of times within nested loops, a C preprocessor macro textually replaces every `min(a,b)` instance with an inline ternary operator before compilation. Adopting the macro, we are able to eliminate function call latency, as reported in Table 9.

Compiler Setup	Matrix Size (n)	Function (s)	Macro (s)
GCC [No Flags]	2000	15.30	13.48
	3000	51.25	45.43
GCC [-O3 -xHost]	5000	7.38	7.30
	8000	30.81	30.21

Table 9: Performance impact of replacing the standard `fmin()` function with a preprocessor macro.

As observed for the Register Promotion analysis, also in this scenario the impact of the macro is significant when the compiler is `gcc` and the optimization flags are disabled. When optimizations flags are enabled or advanced compilers such as `icx` are considered, the standard function call for the minimum computation is automatically replaced with inline assembly instructions. To avoid redundancy, these results have been omitted.

2.5 Results & Comparison

To rigorously validate the quality of our sequential optimizations, the execution time of our optimized sequential implementation (`matrixmult_opt.c`) was compared with the naive (`matrixmult_naive.c`) and library (`matrixmult_library.c`) versions, across various matrix sizes.

Matrix Size (n)	**Naive (s)**	**Optimized (s)**	**CBLAS Library (s)**
1000	0.07	0.04	0.03
2000	0.60	0.36	0.26
3000	2.05	1.20	0.88
5000	10.13	6.02	4.06
8000	39.28	22.51	16.60
10000	76.98	44.37	32.73
15000	259.15	171.66	109.43

Table 10: Execution time comparison between the naive baseline, the optimized implementation, and the CBLAS library across different matrix sizes n, scaling up to 15000. Results were obtained using the best configuration: ICX compiler with `-O3 -xHost` flags.

The Table 10 first highlights the substantial impact of our sequential optimizations against the naive baseline. Across all matrix sizes, the optimized implementation reaches a consistent speedup, reducing the execution time by approximately 35% to 40%. Since both versions were compiled using identical optimization flags (`-O3 -xHost`), this performance gain is attributable to the effect of the Loop Tiling technique.
Despite these optimal results, as expected, our implementation reveals a performance gap compared to the professional CBLAS library which consistently outperforms our version, widening the gap at larger scales. This discrepancy can be justified by the more sophisticated techniques used by the CBLAS library and described in Section 2.2.3.
Subsequently, through the Figure 4, we can visualize the trend of the three implementations, enhancing the performance difference discussed.

2.5.1 Roofline Model

The Roofline Model, illustrated in Figure 5, presents the performance across the different sequential versions implemented for the matrix multiplication problem. Starting from the naive implementation (circle marker), it exhibits a low arithmetic intensity (0.20 FLOP/Byte) and remains strictly memory-bound, constrained primarily by the L2 cache bandwidth. The implementation of loop tiling (triangle marker) in the optimized version effectively enhances the benefits given by the cache locality, shifting the bottleneck to the L1 bandwidth and nearly doubling the throughput (from 27.17 to 48.70 GFLOPS). Ultimately, the CBLAS reference library (square marker) demonstrates the best overall performance. By achieving the highest arithmetic intensity (0.75 FLOP/Byte), it successfully transitions into a compute-bound situation, fully saturating the hardware's execution units.

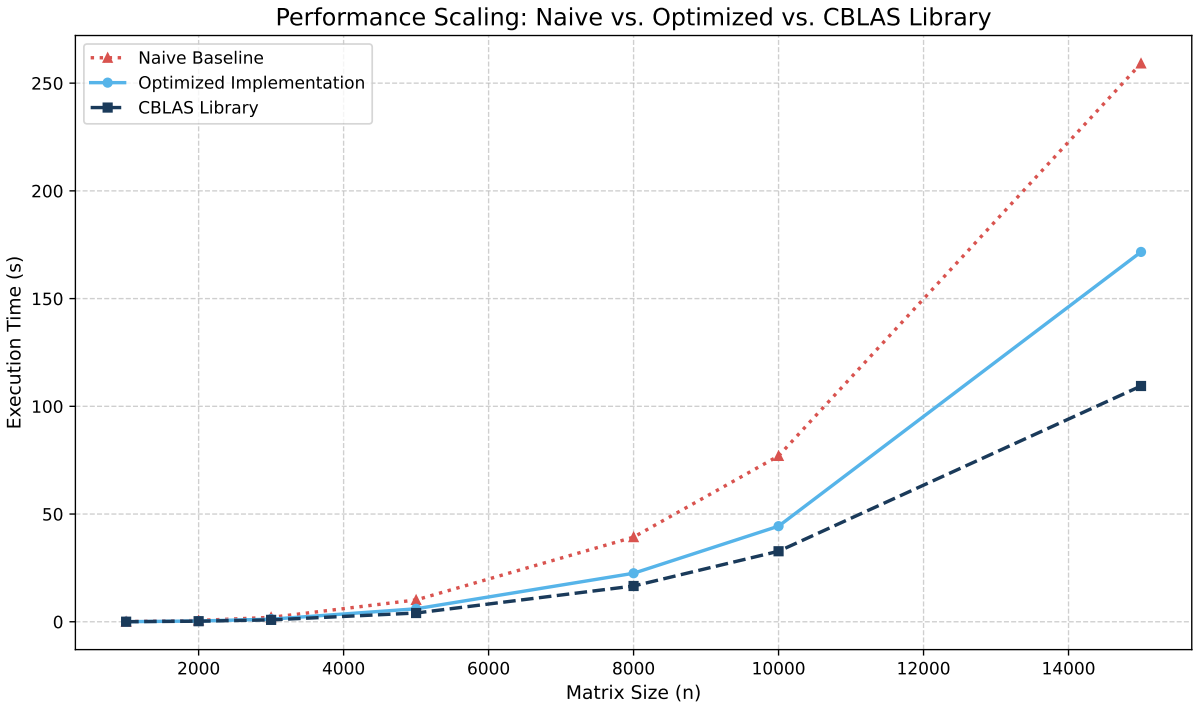

Figure 4: Performance scaling comparison between the naive baseline, the custom optimized implementation and the CBLAS reference library.

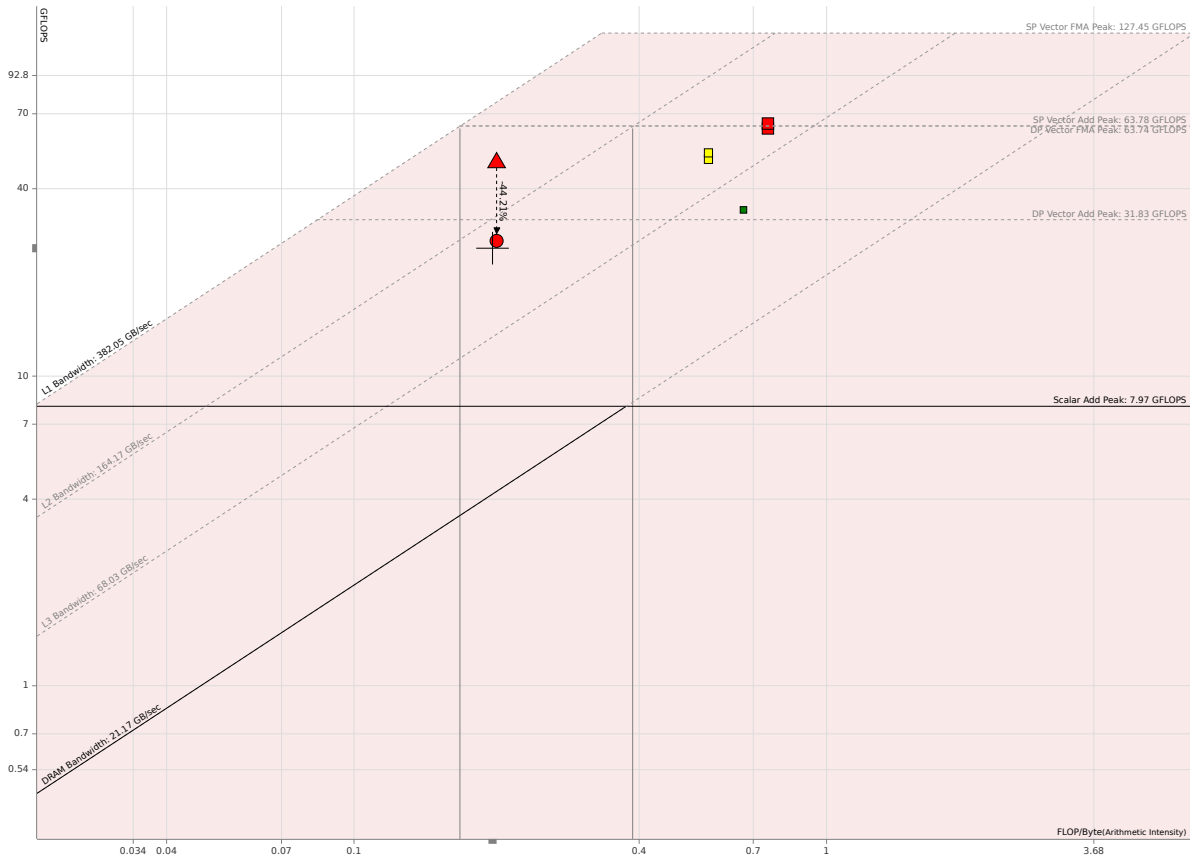

Figure 5: Roofline model illustrating the performance of the three matrix multiplication implementations. The circle, triangle, and square markers denote respectively the naive version, the optimized, and the CBLAS reference library.

3 Shared Memory: OpenMP

After having established an optimized sequential version, the next logical step is to explore and exploit thread-level parallelism through OpenMP (Open Multi-Processing).
The primary objective in this section is to distribute the computational workload of the matrix multiplication across multiple cores of our architecture.
Transitioning from a sequential to a parallel implementation requires a deep analysis of the nested loop structure to identify the most strategic parallelization technique, taking into account data dependencies, variable scoping, and workload distribution to minimize synchronization overhead and prevent race conditions. Following the methodology established in the sequential analysis, this section first evaluates the parallelization of the naive version, and subsequently explores the optimal OpenMP implementation.
As established in the sequential analysis, the optimal compiler configuration identified through our ablation study consists on `icx` compiler with the `-O3 -xHost` flags. Consequently, all OpenMP implementations are evaluated maintaining this configuration, extended only by the `-qopenmp` flag to activate OpenMP directives and enable parallel execution. Additionally, the `include <omp.h>` header is included within the source code to provide access to the OpenMP runtime API and timing functions.
The section concludes with a comparative analysis, evaluating the parallel scalability and the overall performance gains achieved through thread-level parallelism.

3.1 Implementations

Across all OpenMP implementations, two fundamental OpenMP functions are consistently utilized to manage the parallel execution: `omp_set_num_threads()` and `omp_get_wtime()`. The first is crucial for explicitly define the number of threads allocated for each run. The second is strictly required to accurately measure the execution time in a multi-thread environment.

3.1.1 Parallelized Baseline

```
// Matrix multiplication making explicit the policy scheduling
#pragma omp parallel for schedule(guided)
  for (int i = 0; i < n; ++i) {
     for (int k = 0; k < n; k++) {
        for (int j = 0; j < n; ++j) {
           c[i][j] += a[i][k] * b[k][j];
        }
     }
  }
```

Listing 7: OMP Parallelized version of the baseline code

The naive implementation was parallelized utilizing the standard OpenMP directive for parallelization: `pragma omp parallel for`; the mechanism behind of this directive follows the *fork-join* execution model, spawning a team of threads to concurrently execute the iterations of the outermost loop. Furthermore, the scheduling policy was explicitly configured as `guided`; as demonstrated by the ablation study detailed in Section 3.3, this specific configuration was identified as the optimal strategy.

3.1.2 Optimized Version

```
// Matrix multiplication adopting loop tiling technique, collapse
    directive and a specific scheduling policy
#pragma omp parallel for schedule(dynamic, 4) collapse(2)
    // External loops
    for (int ii = 0; ii < N; ii += BLOCK_SIZE) {
        for (int jj = 0; jj < N; jj += BLOCK_SIZE) {
            for (int kk = 0; kk < N; kk += BLOCK_SIZE) {
                int i_end = MIN(ii + BLOCK_SIZE, N);
                int j_end = MIN(jj + BLOCK_SIZE, N);
                int k_end = MIN(kk + BLOCK_SIZE, N);
                // Internal loops: computation in i-k-j order
                for (int i = ii; i < i_end; i++) {
                    for (int k = kk; k < k_end; k++) {
                        for (int j = jj; j < j_end; j++) {
                            c[i][j] += a[i][k] * b[k][j];
                        }
                    }
                }
            }
        }
    }
```

Listing 8: OMP Optimized version.

In addition to adopting the Tiling technique and the OpenMP directive `pragma omp parallel for` for the parallel execution, this version exploits also the OpenMP directive `collapse(2)`. In this matrix multiplication block, the two outermost loops (ii and jj) define the iteration space of the result matrix C, and using collapse(2), we fuse these two loops into a single and larger iteration space.
Without this approach, parallelizing only the outermost loop could lead to *workload imbalance*, particularly for small matrix sizes executed through several threads. In contrast, collapsing the loops into a single iteration space significantly increases the available parallelism, keeping all threads saturated and preventing starvation.
Furthermore, in this specific case, it is important to notice the shift in the scheduling strategy compared to the naive implementation: as documented in Section 3.3.1, this optimized version adopts `schedule(dynamic, 4)`, which represents the optimal choice.

3.1.3 Library Reference: CBLAS

Adopting the same schema of the sequential section, the performance of our parallel implementations were evaluated and compared against the implementation of the CBLAS library (omp_matrixmult_library.c), once again based on the `cblas_dgemm` routine.
From an implementation perspective, the source code remains structurally identical to its sequential counterpart, since the transition to a multi-thread execution model does not require altering the `cblas_dgemm` invocation. The parallelization is achieved at the compiler and runtime levels, by simply including the `<omp.h>` header and adopting the appropriate OpenMP compiler flags. Consequently, the library dynamically manages thread spawning and workload distribution across the available cores during execution.

3.2 Additional Code Optimizations

3.2.1 Restrict keyword

In addition to memory alignment, an approach already adopted and explored for the previous Sequential section, the proposed implementation exploits the keyword `restrict`, which guarantees to the compiler that a specific memory region is accessed only through one pointer. This eliminates any possible risk about memory overlapping (pointer aliasing), allowing the compiler to safely speed up the code using SIMD (Single Instruction, Multiple Data) instructions. Without `restrict`, the compiler could conservatively assume that the memory regions of matrices A, B, and C might overlap, forcing the compiler to reload data from memory frequently to ensure correctness.

3.2.2 OpenMP Scheduling Strategies

The performance of a parallel execution is influenced by how iterations are partitioned and distributed among the available threads. The primary goal of scheduling is to achieve an optimal load balance, ensuring that all cores remain busy in the computation and preventing "idle time", when threads wait for others to finish. In OpenMP, this is managed through the `schedule(type, chunk_size)` clause. Specifically, the `type` parameter defines the workload distribution algorithm and we are able to identify three different types:

- **Static:** the iteration space is divided into blocks of size *chunk_size* and assigned to threads before the loop execution begins. Due to its deterministic nature, it has the lowest overhead and it is ideal when the workload per iteration is perfectly uniform.
- **Dynamic:** iterations are divided into chunks and assigned dynamically to threads: when a thread finishes its chunk, it dynamically requests the next available one. While this introduces higher synchronization overhead, it is highly effective at mitigating *load imbalance* caused by unpredictable execution times.
- **Guided:** this consist on a hybrid approach where the chunk size decreases as the computation progresses. Initially, large chunks are assigned to reduce overhead, while smaller chunks are then used for load balance. The *chunk_size* parameter here represents the minimum size of the final blocks.

Instead, the `chunk_size` parameter determines the number of contiguous iterations treated as a single unit of work to each thread. Selecting an optimal value is fundamental to balance the computation: a small chunk size can increase the frequency of thread synchronization leading to high overhead, while a large chunk size minimizes scheduling overhead and favors cache reuse, but risks significant load imbalance.

3.3 Ablation Study

This section presents the experimental ablation study conducted to identify the optimal OpenMP scheduling configuration. Initially, the evaluation focuses on determining the most effective kind of scheduling policy for the specific workload given by the matrix multiplication computation.
As evidenced by 11, the `guided` scheduling policy emerges as the optimal choice for both

implementations. However, the impact of this advantage varies significantly: while for the naive implementation the performance improvement obtained is significant, for the optimized version it is marginal. Since `guided` and `dynamic` schedules present nearly identical execution times, a secondary ablation study was conducted to investigate and directly compare these two policies by fine-tuning their internal chunk size parameter, aiming to establish the definitive optimal configuration.

Size (n)	**Naive (s)**			**Optimized (s)**		
	Static	**Dynamic**	**Guided**	**Static**	**Dynamic**	**Guided**
8000	15.00	15.76	**10.56**	3.65	**3.37**	3.38
10000	28.97	27.64	**21.19**	7.16	6.66	**6.63**
15000	98.71	111.07	**71.87**	23.31	22.30	**22.24**
20000	264.50	275.24	**171.55**	56.89	53.52	**53.37**

Table 11: Performance evaluation of OpenMP scheduling policies (`static`, `dynamic`, `guided`) across varying matrix sizes for 24 threads.

As indicated by the results of the second ablation study (Table 12), the `dynamic` scheduling policy with a chunk size of 4 emerges as the optimal configuration; consequently, this setup is adopted for all subsequent tests and evaluations.
The chunk parameter explicitly defines the granularity of the workload distribution, representing the specific number of contiguous loop iterations assigned to each thread. This specific chunk size offers the ideal trade-off: it is small enough to keep all threads busy, preventing load imbalance, and also large enough to avoid the overhead of assigning tasks too frequently.

Chunk Size	**Dynamic (s)**	**Guided (s)**
4	**21.89**	22.20
8	22.39	22.36
16	22.55	22.33
32	22.47	22.44
64	22.73	22.69
128	23.51	23.10
256	23.86	24.00

Table 12: Ablation study on the OpenMP chunk size parameter for `dynamic` and `guided` scheduling policies using 24 threads. Matrix size is fixed at $n = 15000$.

Specifically, the results can be analyzed from an architecture perspective. Using the hybrid architecture with 8 high-performance P-Cores (16 threads) and 8 efficiency E-Cores (8 threads) creates a difference in performance between the 24 logical threads. In fact, in our tests `static` scheduling results inefficient because it assigns equal workloads to all threads regardless of their processing power, forcing fast P-Cores to remain idle while waiting for the slower E-Cores to finish. By adopting `dynamic` or `guided` scheduling, we

successfully mitigated this bottleneck. These policies allow faster threads to dynamically "steal" work units as they complete their tasks, ensuring that all hardware resources are saturated and significantly improving parallel efficiency.

3.4 Results & Comparison

To conclude the section, using the optimal configuration identified in the preceding ablation study, a rigorous strong scaling analysis by varying the number of threads is conducted. Successively, to accurately contextualize the value of the results of our optimized version, the execution times are compared against the naive baseline and the CBLAS library, which establishes the architectural upper bound.
Starting from the results presented in Table 13 and in Figure 6, a clear correlation immediately emerges between the number of OpenMP threads and the reduction in execution time. As expected, the most substantial performance gains are achieved exploiting the hardware's concurrent capabilities. In detail, the best performance are obtained when the workload is distributed across the maximum number of available threads (24).

Num. Threads	**Naive Baseline (s)**	**Optimized Version (s)**
4	65.88	18.02
8	33.22	9.32
12	25.06	7.59
16	21.67	6.97
24	**21.40**	**6.58**

Table 13: Scaling comparison between the naive baseline and the optimized OpenMP implementation for a fixed matrix size of $n = 10000$.

As observed in Figure 6, the execution time shows a rapid decrease up to 16 threads, followed by a flattening of the scaling curve between 16 and 24 threads. This behavior demonstrates that the workload effectively saturates the 16 physical cores, making marginal the benefits obtained by the addition of further logical threads. In any case, the trade-off of thread-level parallelism must be taken into account: adding more threads is not always the optimal solution, since it involves the costs associated to scheduling overhead and thread synchronization.
Moving to the final comparison, Table 14 and Figure 7 illustrate the performance of our optimal implementation against both the naive baseline and the CBLAS reference.
The data reveal a consistent enhancement over the baseline, validating the efficacy of our custom optimizations, while CBLAS library maintains a distinct superiority, which becomes increasingly pronounced as the matrix size scales. Consistent with the sequential analysis, the underlying architectural reasons for this gap remain identical: professional libraries meticulously exploit the entire memory hierarchy through advanced techniques and direct assembly-level instructions.

3.4.1 Roofline Model

Figure 8 illustrates the multi-thread Roofline analysis for the OpenMP implementations, highlighting the performance scaling achieved through parallelization and memory opti-

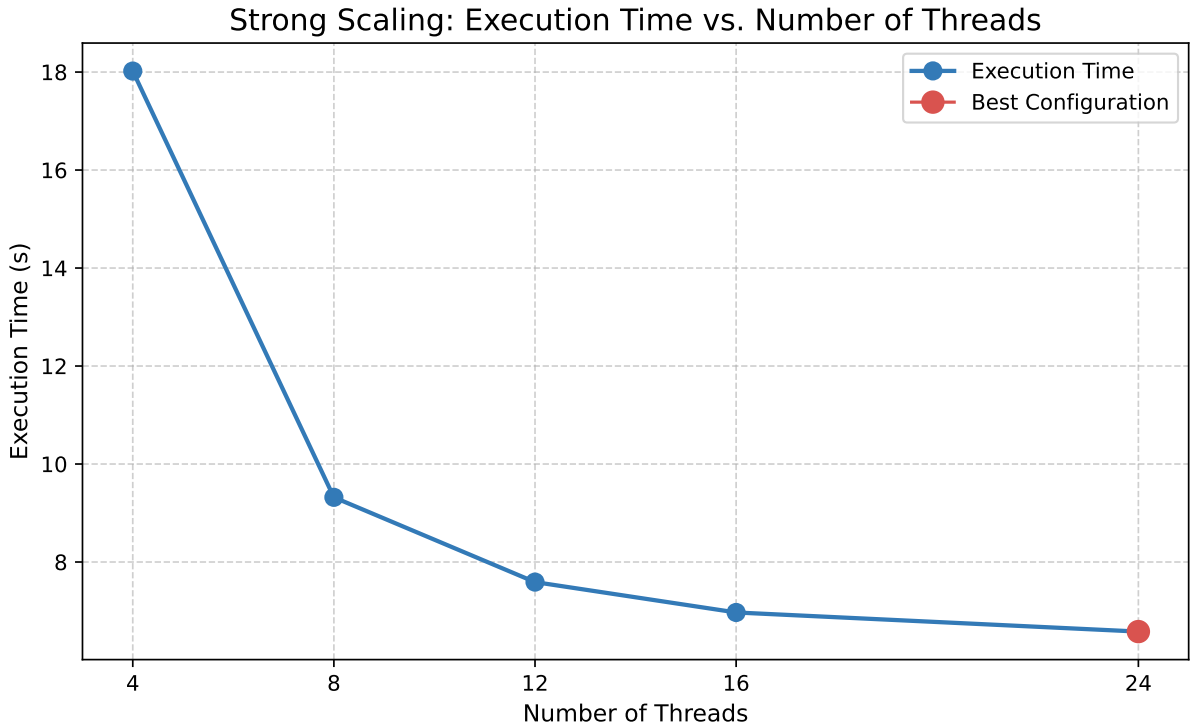

Figure 6: Scaling analysis of the optimized OpenMP implementation utilizing the best-performing scheduling policy (`dynamic`, chunk size 4) for a fixed matrix size of $n = 10000$. The red marker highlights the optimal configuration, achieving the absolute minimum execution time at 24 threads.

Matrix Size (n)	**Naive (s)**	**Optimized (s)**	**Library (s)**
8000	10.67	3.52	2.11
10000	20.95	6.79	4.19
15000	73.38	22.49	12.56
20000	174.05	54.13	31.65

Table 14: Execution time comparison among the naive baseline, OpenMP optimized version, and CBLAS library across varying matrix sizes, utilizing 24 threads.

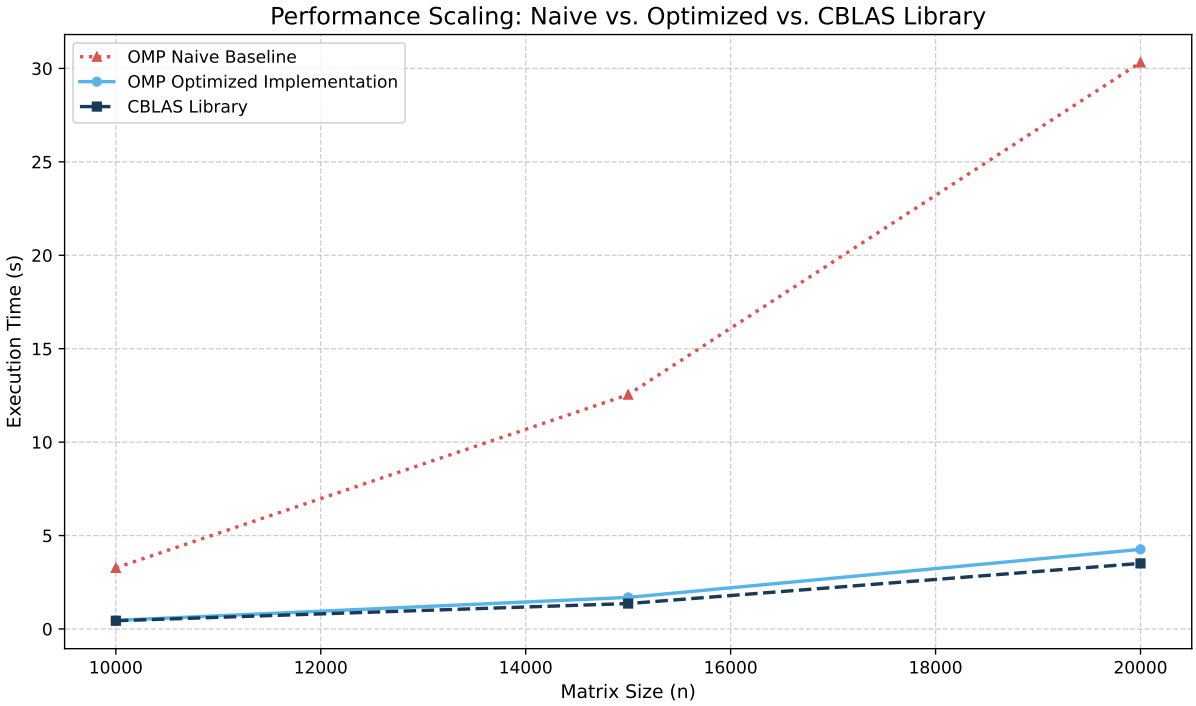

Figure 7: Visualization of the comparative performance between the three OpenMP implementations. Execution times are evaluated across scaling matrix sizes ($n = 8000$ to 20000) utilizing a fixed configuration of 24 threads.

mizations.
Starting, as always, from the naive baseline (square marker), it operates at a low arithmetic intensity ($\approx$ 0.1 FLOP/Byte) and remains severely memory-bound; despite the utilization of multiple threads, its throughput is constrained due to poor cache locality. Applying the loop tiling strategy (circle marker), it effectively increases data reuse, shifting the arithmetic intensity to approximately 0.25 FLOP/Byte. In particular, the optimized implementation is represented by two circles with identical arithmetic intensity but divergent computational throughputs: the upper circle ($\approx$ 250-300 GFLOPS) represents the efficient inner computational loops, while the other circle ($\approx$ 100 GFLOPS) corresponds to the outer parallel regions, reflecting the inevitable overhead of OpenMP thread scheduling and synchronization.
Finally, as seen for the sequential analysis, the CBLAS reference library (triangle marker) establishes the empirical upper bound of the hardware, achieving an optimal arithmetic intensity of $\approx$ 0.7 FLOP/Byte and allowing the library to be in a compute-bound regime.

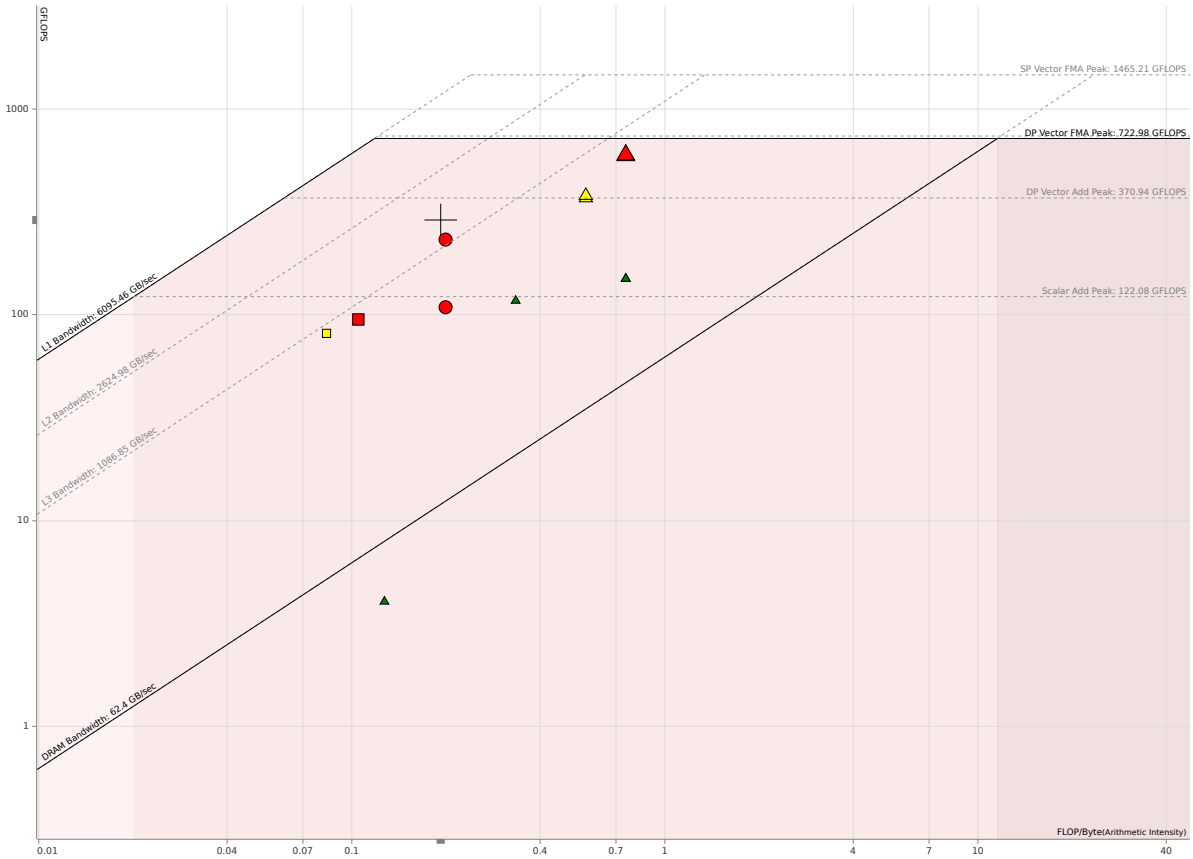

Figure 8: Roofline model illustrating the performance of the three evaluated OpenMP matrix multiplication implementations. The square, circle, and triangle markers denote the naive baseline, the custom optimized version, and the CBLAS reference library, respectively.

4 Distributed Computing: MPI

After the development of a shared memory parallelized implementation through OpenMP, the following section proceeds to tackle a distributed memory version through MPI.
While remaining within the SPMD paradigm, MPI distances itself from the implicit synchronization of fork-join by requiring explicit management of all process interactions. In MPI, each process is assigned a unique identifier called rank; throughout this section, the terms process and rank will be used interchangeably. This highlights the main challenge of the MPI parallelization process: designing an appropriate communication layout. Consistently with earlier sections, this chapter starts by exploring a naive parallelization of the baseline code. Then, a new and more communication efficient algorithm (SUMMA) is briefly introduced along with our corresponding implementation. Finally, the section concludes with a comparative analysis between the naive version, our SUMMA-based optimization and a state-of-the-art library (ScaLAPACK).

4.1 Implementations

Before presenting the individual MPI implementations, a note on initialization is required. The test matrices adopted in this study are constant; in principle they could be generated independently by each process but we deliberately chose a simpler but more general strategy: the process with rank 0 allocates and initializes the full A, B, and C matrices, and only afterwards distributes the required portions to the other ranks. This approach is inefficient in terms of memory usage, but it is also more general, since it would remain applicable for non-constant input data. In production environments, a more scalable solution would instead rely on distributed I/O, with each rank loading only its own portion of the matrices from external storage, thus avoiding the presence of the full data on a single process.
This design choice does not affect the reported performance results, since timing starts only after initialization through `MPI_Wtime()`. It does, however, explain why the largest matrix size considered in this section is limited to $n = 15000$, as larger inputs would exhaust the available RAM on rank 0.

4.1.1 Parallelized Baseline

Communication strategy
In this naive load balancing strategy each rank is responsible for the computation of a portion of C rows. To reach this goal, each rank is delivered an equal share of A rows and the whole B matrix. The simplicity of this approach is balanced by its evident wastefulness: since MPI is a distributed memory approach, sharing B to each process means duplicating the memory space it inhabits.

```
int rows_per_proc = (n / size) + (rank < remainder ? 1 : 0);
MPI_Barrier(MPI_COMM_WORLD);
MPI_Bcast(b, n * n, MPI_DOUBLE, 0, MPI_COMM_WORLD);
MPI_Scatterv(a, sendcounts, displs, MPI_DOUBLE,
             local_a, rows_per_proc * n, MPI_DOUBLE,
             0, MPI_COMM_WORLD);
```

Listing 9: A naive approach to MPI data distribution

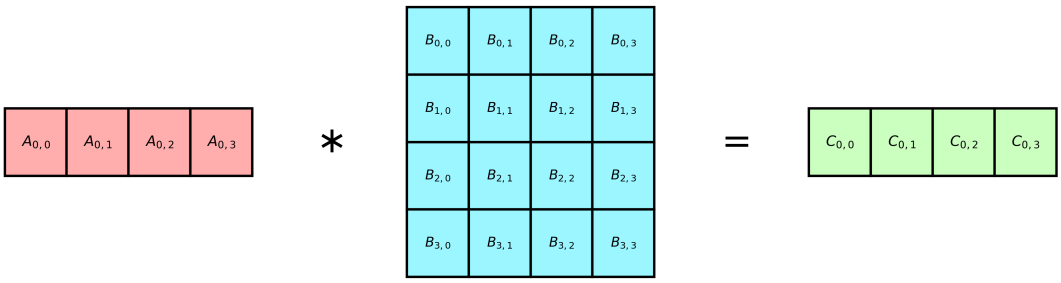

Figure 9: Schematic illustration of the computation performed by the process at rank 0 in the naive MPI implemetation. In this demonstrative case both the matrix size and the number of processes equals 4, so each rank computes a single row of C.

Local computation

As illustrated in figure 9, each rank computes its assigned partition of rows locally. As in 1, the calculation follows an IKJ triple-nested loop, serving as an barely optimized baseline that performs the dot product between the local rows of A and the broadcasted matrix B.

```
for (int i = 0; i < rows_per_proc; i++) {
    for (int k = 0; k < n; k++) {
        double temp = local_a[i * n + k];
        for (int j = 0; j < n; j++) {
            local_c[i * n + j] += temp * b[k * n + j];
        }
    }
}
```

Listing 10: Local computation performed by each rank in the Naive MPI implementation

4.1.2 Optimized Version

Our optimized implementation is based on SUMMA (Scalable Universal Matrix Multiplication Algorithm)[3], the industry standard for distributed matrix multiplication algorithms. It organizes processors into a 2D grid and divides each matrix into sub-matrices (panels). This strategy minimizes memory consumption by replacing global broadcasts with localized panel-wise updates. Iteratively, only the necessary panels of A and B are moved across the grid rows and columns, allowing each processor to compute its C sub-matrix.

Communication strategy

Following the panel-based decomposition, the processes are mapped to a symmetric 2D Cartesian grid. Consequently, our implementation is strictly bound to a perfect square number of processes to maintain a perfectly uniform block distribution.

To support a more carefully targeted broadcast, row-specific and column-specific broadcast communicators are built, providing each process the exact panels needed to compute its C sub-matrix.

```
MPI_Comm cart_comm, row_comm, col_comm;

MPI_Cart_create(MPI_COMM_WORLD, 2, dims, periods, 0, &cart_comm);
MPI_Cart_coords(cart_comm, rank, 2, coords);

int my_row = coords[0];
int my_col = coords[1];

MPI_Comm_split(cart_comm, my_row, my_col, &row_comm);
MPI_Comm_split(cart_comm, my_col, my_row, &col_comm);
```

Listing 11: Building cartesian row-exclusive or column-exclusive communicators

SUMMA local computation

Each of the q processes, identified by rank k and indices (i,j), is responsible for the computation of the C submatrix

$$C_{i,j} = \sum_{k=0}^{q-1} A_{i,k} \times B_{k,j}, \tag{1}$$

which is visually summarized in Figure 10.

Code snippet 12 shows that this summation is physically realized by the outermost k-loop. At each iteration, the processes use their grid coordinates (my_row and my_col) to determine if they own the specific $A_{I,K}$ or $B_{K,J}$ block. After, the targeted broadcasts distribute these blocks into a_panel and b_panel. The inner tiled loops (as in the optimized sequential implementation 2) perform the local matrix multiplication, accumulating the partial products into the $C_{I,J}$ sub-matrix using the += operator.

```
for (int k = 0; k < p_cols; k++) {
    if (my_col == k) // Row broadcast to get block for A
        memcpy(a_panel, a_local, block_size * sizeof(double));
    MPI_Bcast(a_panel, block_size, MPI_DOUBLE, k, row_comm);
    if (my_row == k) // Column broadcast to get block for B
        memcpy(b_panel, b_local, block_size * sizeof(double));
    MPI_Bcast(b_panel, block_size, MPI_DOUBLE, k, col_comm);

    // Tiled local multiplication
    for (int ii = 0; ii < local_rows; ii += NB) {
        for (int kk = 0; kk < local_cols; kk += NB) {
            for (int jj = 0; jj < local_cols; jj += NB) {
                int i_max = MIN(ii + NB, local_rows);
                int k_max = MIN(kk + NB, local_cols);
                int j_max = MIN(jj + NB, local_cols);
                // internal loop
                for (int i = ii; i < i_max; i++) {
                    for (int k_idx = kk; k_idx < k_max; k_idx++)
                        {
```

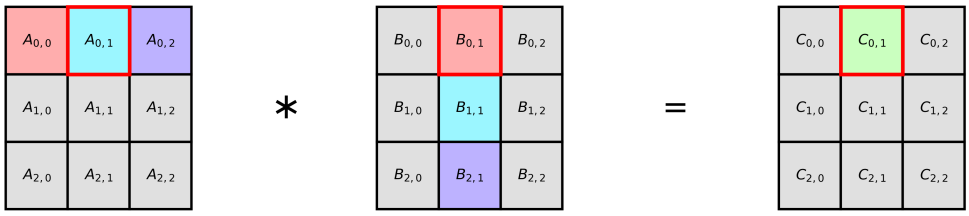

Figure 10: Schematic representation of the iterative process behind the computation of $C_{0,1}$ by process (0,1).
The inner cells highlighted in red ($A_{0,1}$ and $B_{0,1}$) are owned by process (0,1), while the other colored cells are iteratively broadcasted from the other processes through the row/-column communicators.

```
                                    double a_ik = a_panel[i * local_cols +
                                        k_idx];
                                    for (int j = jj; j < j_max; j++) {
                                        c_local[i * local_cols + j] += a_ik *
                                            b_panel[k_idx * local_cols + j];
                                    }
                                }
                            }
                        }
                    }
                }
            }
}
```

Listing 12: Local computation of cell $C_{i,j}$. The tile size NB is set as 64, a value validated as optimal by the sequential ablation study.

4.1.3 Library reference: ScaLAPACK

As in the previous sections, we benchmark our SUMMA-based optimization against a state-of-the-art reference: ScaLAPACK (`matrixmult_mpi_library.c`). While both approaches share the foundational SUMMA logic, ScaLAPACK introduces several advanced architectural layers to achieve peak performance:

- **2D Block-Cyclic Distribution:** It partitions the matrices into smaller, scattered tiles rather than simple contiguous blocks (as seen in figure 11), guaranteeing optimal load balancing across the processor grid.
- **BLACS Integration:** The communication overhead is minimized through Basic Linear Algebra Communication Subprograms (BLACS), a specialized commu-

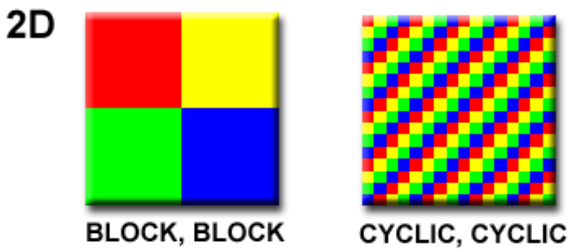

Figure 11: Intuitive representation of block and block-cyclic distribution. The former was used by our SUMMA-based implementation, while the latter is adopted by ScaLAPACK.

nication layer designed to overlap communication and computation while avoiding blocking calls.

- **Underlying BLAS Routines:** ScaLAPACK interoperates with a FORTRAN version of the already mentioned highly optimized BLAS functions (here called `pdgemm_`) for the local node-level computation, reaching near-optimal node performance.

```
MPI_Barrier(MPI_COMM_WORLD);
double start = MPI_Wtime();
pdgemm_("N", "N", &n, &n, &n, &d_one, local_a, &i_one, &i_one,
   descA,local_b, &i_one, &i_one, descB,&d_zero,local_c, &i_one, &
   i_one, descC);
MPI_Barrier(MPI_COMM_WORLD);
double end = MPI_Wtime();
```

Listing 13: ScaLAPACK main routine

4.2 Ablation study

This ablation study determines the optimal process count, testing the four perfect-square configurations compatible with the 2D grid constraint: 4, 9, 16, and 25 processes. The critical analysis evaluates the trade-off between 16 processes (the largest perfect square supported by the physical cores) and a 25-process setup (which oversubscribes the system by one core). As figure 12 clearly shows, the scaling behaviors diverge with the introduction of oversubscription and the consequential necessity of time-share a physical core.

4.2.1 Oversubscription is beneficial for the optimized version

The optimized version improves the execution time from 27.14s (16 processes) to 25.84s (25 processes). This gain is to be attributed to a positive trade-off between context-switching and having smaller sub-matrices:

- Shrinking local matrix blocks on a 5×5 grid drastically reduces the number of operations each process executes, leveraging the $O(n^3)$ scaling of matrix multiplication.
- While optimized via tiling, the tiled inner loops do not push the physical cores to their absolute theoretical limits.

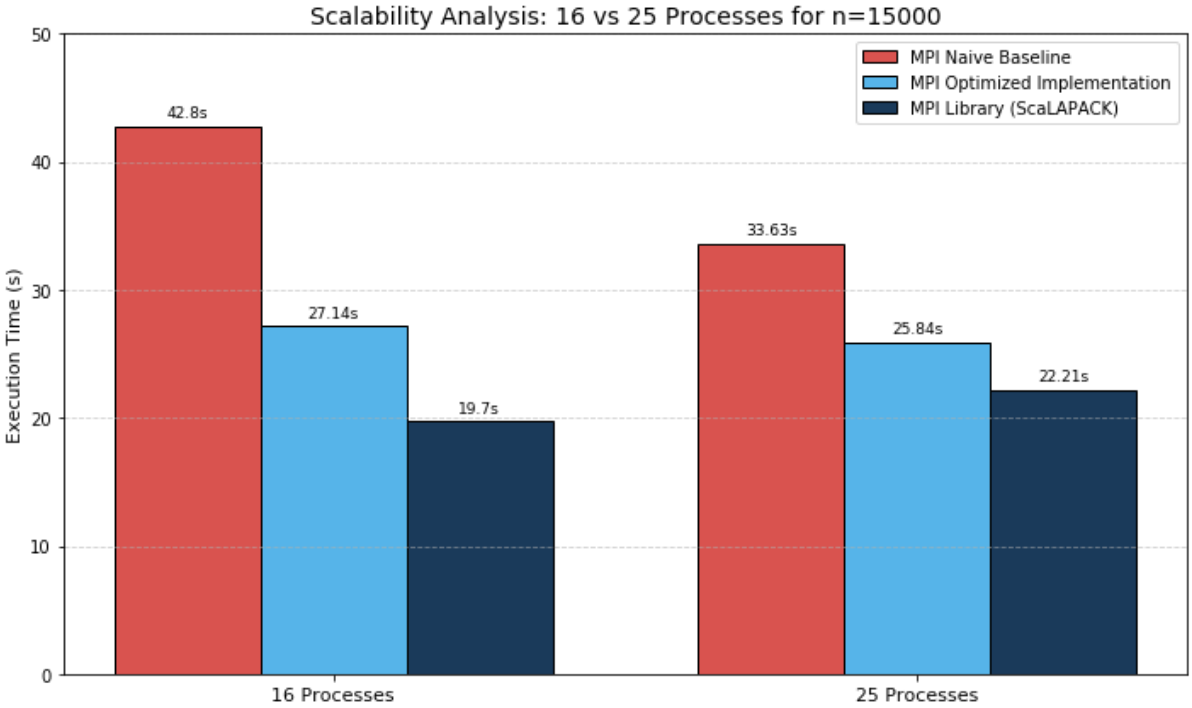

Figure 12: Performance comparison of the naive, optimized, and library implementations scaling from 16 to 25 processes.

- This lack of saturation leaves the operating system enough bandwidth to manage the context-swith between the two oversubscribed processes on the shared core with a relatively low penalty.

To summarize, the scheduling penalty introduced by oversubscription is more than compensated by the reduced workload assigned to each process, resulting in an overall faster execution.

4.2.2 Oversubscription harms the library's performance

In contrast, ScaLAPACK's execution time degrades from 19.7s to 22.21s when transitioning from 16 to 25 processes. This penalty is caused by the oversubscribed core breaking the library's synchronization pipeline. There are two main arguments explaining this degradation:

- The internal BLAS routines are designed to saturate CPU resources. Forcing two processes to share one physical core causes violent resource competition, nearly doubling their local computation time.
- The underlying BLACS layer, paired with the block-cyclic distribution, requires frequent global synchronization. When in need of a tile under the responsibility of the oversubscribed core, fast processes on dedicated cores must sit completely idle at communication barriers.

To summarize, the execution time of such a frequently synchronized program is bound by the slowest, oversubscribed process.

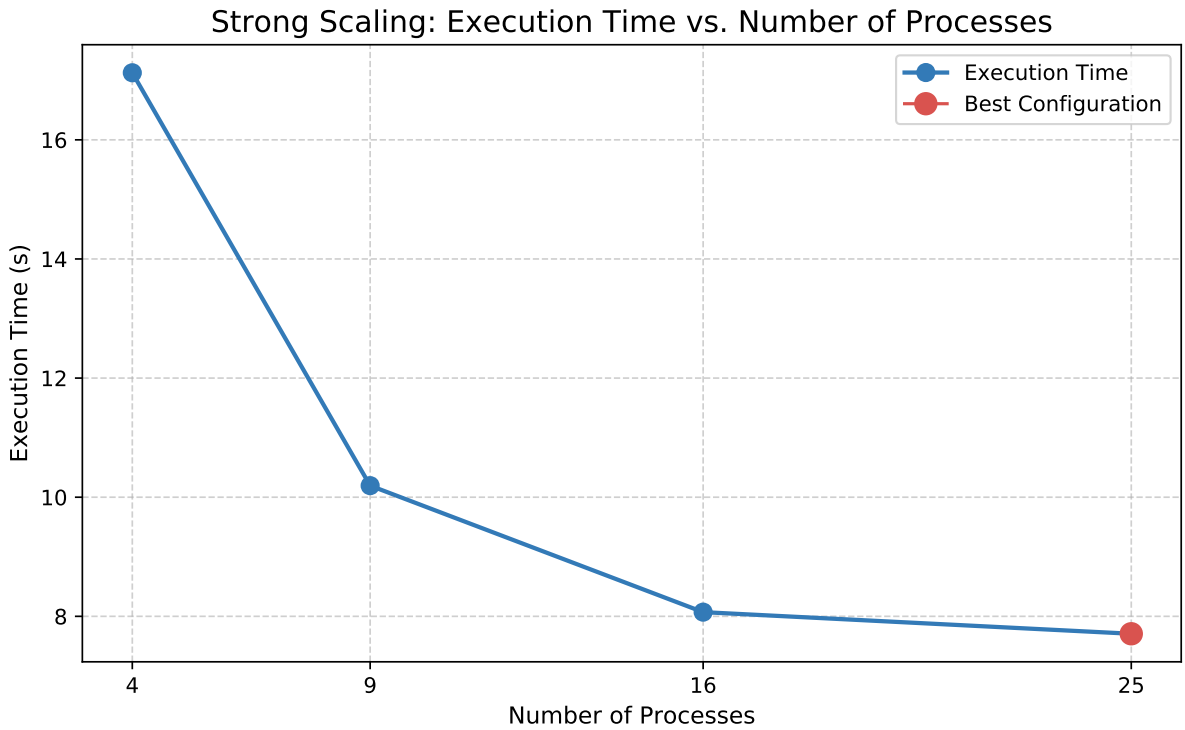

Figure 13: Scaling analysis of the optimized MPI implementation for a fixed matrix size of n = 10000. The red marker highlights the optimal configuration, achieving the absolute minimum execution time at 25 threads

4.3 Results & Comparison

Bringing this section to a close, we first perform a strong scaling analysis by varying the number of processes while keeping the matrix size fixed. After that we finally compare the execution times of our optimized SUMMA implementation against the naive baseline and the ScaLAPACK library.

Num. Processes	Naive Baseline (s)	Optimized Version (s)
4	20.04	17.13
9	10.23	10.19
16	11.69	8.07
25	**10.48**	**7.71**

Table 15: Scaling comparison between the naive baseline and the optimized MPI implementation for a fixed matrix size of $n = 10000$.

As illustrated in table 15 and figure 13, the scaling behaviors aligns with the OpenMP results: execution time decreases up to 16 processes, after which the improvement obtained by moving to 25 processes becomes much smaller. This behavior is to be attributed to the matrix size $N = 10000$, which is simply too small to justify the increased grid complexity.

The final comparison (at table 16 and figure 14) involves the naive baseline, our optimized

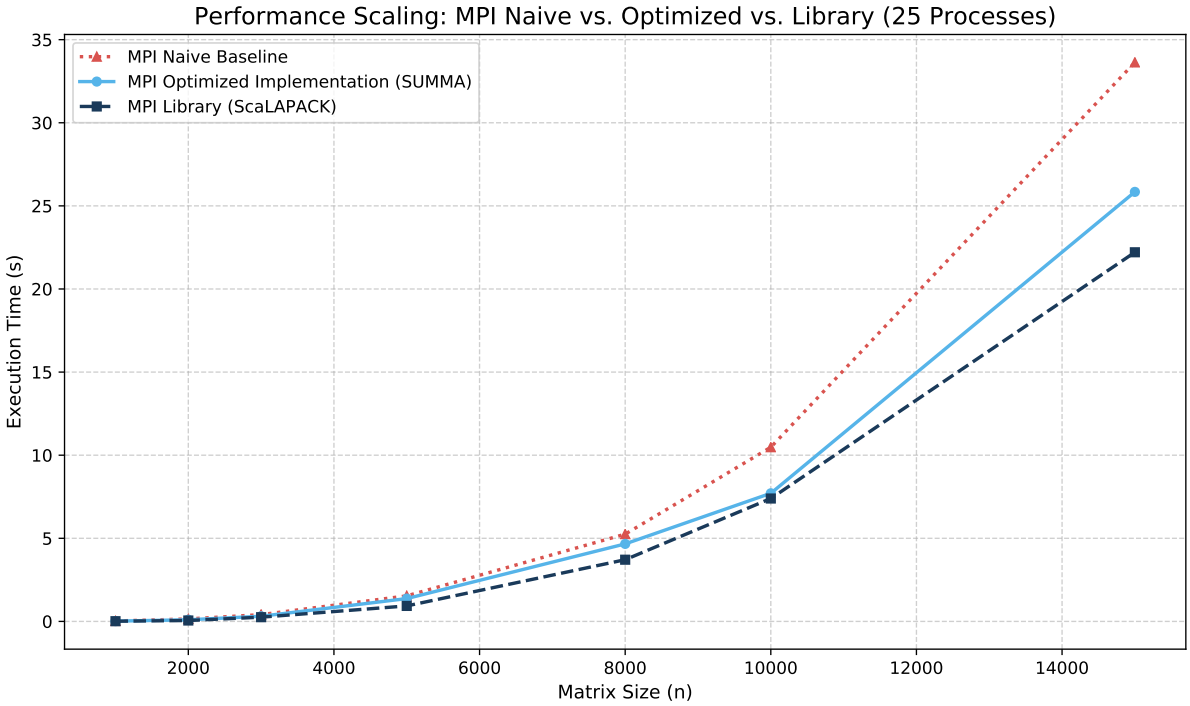

Figure 14: Caption

implementation and the ScaLAPACK version, all executed using a 25-process configuration. As seen in figure 12, such a configuration benefits naive and optimized versions while damaging the library performance. Consequently, this specific hardware configuration allowed our custom implementation to achieve performance parity with the highly tuned ScaLAPACK library.

Matrix Size (n)	**Naive (s)**	**Optimized (s)**	**Library (s)**
5000	1.54	1.37	0.93
8000	5.25	4.66	3.71
10000	10.48	7.71	7.39
15000	33.63	25.84	22.21

Table 16: Execution time comparison among the naive baseline, MPI optimized version, and ScaLAPACK library across varying matrix sizes, utilizing 25 processes.

5 Heterogeneous Computing: CUDA

CUDA (Compute Unified Device Architecture) is NVIDIA's programming model for General Purpose GPU (GPGPU) computing: it transforms the graphics card into a powerful math coprocessor to process massive amounts of data in parallel.
The execution model is based on the collaboration between the Host (CPU, which handles sequential logic) and the Device (GPU , which handles intensive parallel tasks). Describing briefly a typical scenario, the CPU copies data to the GPU memory and launches the "Kernel", a special function executed simultaneously on the GPU by thousands of threads; once the computation is complete, the CPU retrieves the results from the Device Memory. From a software perspective, this interaction is managed through the CUDA Runtime API, which require the inclusion of the `<cuda_runtime.h>` header.
The general layout of the section strictly follows the same pattern adopted in the previously examined chapters.

5.1 Implementations

All CUDA implementations were developed and executed using the Google Colaboratory (Colab) environment, which allows to leverage its dedicated GPU hardware resources for testing and profiling massively parallel workloads. The codes were compiled using `nvcc` (NVIDIA CUDA Compiler), which handles the separation and compilation of the Host code for the standard C/C++ compiler and the Device code for the GPU architecture. In addition, the `-arch=sm_75` and `-O3` flags are used to maximize hardware utilization. Specifically, the architectural characteristics of the GPU are summarized in Table 17.

Hardware Feature	**Specification**
GPU Model	NVIDIA Tesla T4
Microarchitecture	Turing
Compute Capability	7.5 (`sm_75`)
Global Memory (VRAM)	16 GB GDDR6 (15.36 GB Usable)
Memory Bandwidth	320 GB/s
CUDA Cores	2560
Streaming Multiprocessors (SMs)	40
Peak FP32 Performance	$\approx$ 8.1 TFLOPS

Table 17: Hardware specifications and theoretical peak performance of the NVIDIA Tesla T4 GPU utilized in the Google Colab environment for the CUDA benchmarks.

5.1.1 Preamble: the shift to Single Precision

Initially, GPU implementations, as all the previous implementations, maintained the double precision arithmetic to ensure consistency across the evaluated paradigms. However, as demonstrated in Table 18 and detailed further in Section 5.3, the performance achieved through the best configuration, was severely suboptimal.

Matrix Size (n)	Naive (s)	Optimized (s)	Library (s)
10000	10.35	8.04	8.01
15000	38.63	26.85	26.92

Table 18: Execution times of the matrix multiplication implementations using double-precision (FP64) arithmetic.

This performance bottleneck derives directly from the hardware of our Tesla T4 GPU; its Turing micro-architecture is highly optimized for single-precision arithmetic (FP32) rather than double-precision arithmetic (FP64). Specifically, within each Streaming Multiprocessor, there is only one FP64 core for every 32 FP32 cores, creating a consistent unbalance in performance. We can quantify this in terms of throughput: the GPU reaches a maximum peak performance of 8.1 TFLOPS in FP32, while for FP64 it drops vertiginously, achieving approximately 0.25 TFLOPS.
Hence, to fully exploit our GPU's resources to achieve competitive execution times, all variables and matrices are converted to single precision (float), maintaining an acceptable degree of numerical accuracy for the scope of the project.

5.1.2 Parallelized Baseline

```
// Global function: callable from Host, executed on Device
__global__ void matMulNaive(const float * a, const float * b,
   float * c, int n) {

    // Global 2D coordinates for the output matrix C
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    // Matrix multiplication through 1D flat arrays
    if (row < n && col < n) {
        float sum = 0.0f;
        for (int k = 0; k < n; ++k)
            sum += a[row * n + k] * b[k * n + col];
        c[row * n + col] = sum;
    }
}
```

Listing 14: CUDA Kernel of the naive implementation.

The naive CUDA implementation parallelizes the matrix multiplication algorithm by mapping its two outermost loops directly onto the GPU execution grid.
In this first simple approach, each thread is uniquely assigned to compute a single element of the output matrix C, with the global matrix coordinates dynamically calculated using `blockIdx` and `threadIdx` variables.
A boundary check was inserted to prevent invalid memory accesses when the matrix dimension n is not perfectly divisible by the block size.
Consequently, the innermost loop is responsible for the dot product, fetching the operands directly from Global Memory.
However, despite coalesced accesses through 1D linear indices, this naive approach is

severely memory-bound. The computing cores frequently stall waiting for data from the slow Global Memory, highlighting the need for a faster caching strategy.

5.1.3 Optimized Version

```
__global__ void MatMult(int n, const float *A, const float *B,
   float *C) {

    // Shared Memory exploited by the Thread Block
    __shared__ float sA[BM][BK];
    __shared__ float sB[BK][BN];

    // Local coordinates, registers and thread index
    int bx = blockIdx.x;
    int by = blockIdx.y;
    int tx = threadIdx.x;
    int ty = threadIdx.y;
    float res[TM][TN] = {0.0f};
    float a_reg[TM];
    float b_reg[TN];
    int tid = ty * (BN / TN) + tx;

    // Loop: slicing along the k dimension (BK)
    for (int p = 0; p < (n + BK - 1) / BK; ++p) {

        // Decoupled Loading.
        // Each thread transfers contiguous elements from the
           Global to the Shared Memory (A and B).

        // Loading of a slice of the Tile A
        for (int i = 0; i < (BM * BK) / NUM_THREADS; ++i) {
            int idx = i * NUM_THREADS + tid;
            int a_row = idx / BK;
            int a_col = idx % BK;
            int g_a_row = by * BM + a_row;
            int g_a_col = p * BK + a_col;
            sA[a_row][a_col] = (g_a_row < n && g_a_col < n) ? A[
               g_a_row * n + g_a_col] : 0.0f;
        }

        // Loading of a slice of the Tile B
        for (int i = 0; i < (BK * BN) / NUM_THREADS; ++i) {
            int idx = i * NUM_THREADS + tid;
            int b_row = idx / BN;
            int b_col = idx % BN;
            int g_b_row = p * BK + b_row;
            int g_b_col = bx * BN + b_col;
            sB[b_row][b_col] = (g_b_row < n && g_b_col < n) ? B[
               g_b_row * n + g_b_col] : 0.0f;
        }
        __syncthreads();
```

```
        // Register Promotion.
        // From Shared Memory to registers for the computation
        #pragma unroll
        for (int k = 0; k < BK; ++k) {
            for(int i=0; i<TM; i++) a_reg[i] = sA[ty*TM+i][k];
            for(int j=0; j<TN; j++) b_reg[j] = sB[k][tx*TN+j];

            for(int i=0; i<TM; i++) {
                for(int j=0; j<TN; j++)
                    res[i][j] += a_reg[i] * b_reg[j];
            }
        }
        __syncthreads();
    }

    // Saving results in the Global Memory C
    int row = by * BM + ty * TM;
    int col = bx * BN + tx * TN;

    for (int i = 0; i < TM; ++i) {
        for (int j = 0; j < TN; ++j) {
            int c_row = row + i;
            int c_col = col + j;
            if (c_row < n && c_col < n)
                C[c_row * n + c_col] = res[i][j];
        }
    }
}
```

Listing 15: Kernel CUDA of the optimized implementation.

As for the other programming paradigms examined, the primary objective is to achieve performance comparable to the reference cuBLAS library [1]. Since preliminary implementations based on a standard tiling technique failed to close the performance gap, our implementation strategy shifted towards advanced techniques employed by the cuBLAS library, in order to maximize the performance and mitigate the problems described for the naive implementation.
Specifically, the code implements a multi-level memory hierarchy strategy (2D Register Tiling) to overcome the global memory bottleneck. This is based on the following fundamental concepts:

- **Decoupled Loading and Memory Coalescing:** Rather than fetching operands continuously from Global Memory, a Thread Block loads tiles into the low latency Shared Memory; in particular, the fetching phase is completely decoupled from the computation phase: threads temporarily ignore their 2D mathematical assignment and behave as a 1D linear array. This guarantees perfect memory coalescing, allowing the hardware to group multiple requests into single, massive memory transactions. A synchronization barrier (`__syncthreads()`) is fundamental: it ensures all data is safely loaded before the computation begins.

- **Sub-tiling and Register Promotion:** Rather than computing a single output element, each thread is responsible for a sub-tile ($TM \times TN$). Threads pre-load the required data from Shared Memory directly into their fast registers (`a_reg` and `b_reg`) and the partial sums are accumulated locally in the `res` registers.

Lastly, the dot product computation, which uses the data stored in the registers, is is explicitly unrolled using the `#pragma unroll` directive, to eliminate loop overhead and maximize computational efficiency.
All these strategies, inspired by the underlying mechanism of the cuBLAS library, allow our implementation to reach optimal performance level, as documented in Section 5.4.

5.1.4 Library reference: cuBLAS

CuBLAS (CUDA Basic Linear Algebra Subroutines) is a GPU-accelerated library developed by NVIDIA that provides highly optimized routines for linear algebra operations, such as matrix and vector multiplications, directly on the GPU.
While the code strictly regarding matrix multiplication computation (`cublasSgemm` routine) remains structurally consistent with the library versions examined in the previous sections, the overall implementation differs due to the explicit management of the Host and Device environments. The introduction of the CUDA paradigm requires memory allocation, data transfers, and execution synchronization for the two environments.

5.2 Additional Code Optimizations

5.2.1 Data type float4

```
// Vectorized Memory Access
float4 tmpA = make_float4(0.0f, 0.0f, 0.0f, 0.0f);
if (g_a_row < pad_n && g_a_col < pad_n)
    tmpA = *(const float4*)&A[g_a_row * pad_n + g_a_col];
// Vectorized Write in Shared Memory
*(float4*)&sA[a_row][a_col_v] = tmpA;
```

Listing 16: Usage of 4float data type.

Going deeper with the optimizations to further reduce the performance gap with the cuBLAS library, we introduced the Memory Vectorization technique using the `float4` data type. The illustrated code snippet represents the core of this technique and the only significant change with respect to the 2D Register Tiling version: it modifies the Decoupled Loading phase by managing the vectorized data transfer from Global to Shared Memory through an explicit pointer casting.
Instead of using separate 32-bit memory requests for individual floating-point numbers, this technique forces the GPU to fetch four consecutive elements in a single 128-bit memory transaction, reducing instruction overhead and maximizing memory bandwidth utilization.
Furthermore, we integrated a padding strategy on the Host side (rounding the matrix dimensions to the nearest multiple of the block size) in order to ensure the required memory alignment and to maintain the code compatible with matrices of any size.
Adopting this specific optimization, we can further reduce the execution time of our optimized version, significantly closing the performance gap with the cuBLAS library, as documented in Section 5.4.

5.3 Hyperparameter Tuning

In this section is shown the hyperparameter tuning process, necessary to find the optimal configuration. The hyperparameters involved consist of:

- BM, BN: define the dimensions of the Tile, representing the workload assigned to each Thread Block.
- BK: represents the step size along the inner reduction dimension (K), determining the chunk of elements cooperatively loaded into Shared Memory during each computational phase.
- TM, TN: refer to the dimensions of the small thread-level block (sub-matrix of the Tile), determining the number of output elements computed by each thread.

Configuration	**Execution Time (s)**
{`BM,BN=128, TM,TN=8, BK=32`}	1.8973
{`BM,BN=128, TM,TN=8, BK=16`}	1.9237
{`BM,BN=128, TM,TN=8, BK=8`}	1.9717
{`BM,BN=128, TM,TN=4, BK=8`}	2.2403
{`BM,BN=64, TM,TN=4, BK=32`}	2.7345
{`BM,BN=64, TM,TN=8, BK=16`}	2.8113
{`BM,BN=64, TM,TN=8, BK=8`}	2.8712
{`BM,BN=64, TM,TN=8, BK=32`}	2.8865
{`BM,BN=64, TM,TN=4, BK=8`}	2.9866
{`BM,BN=64, TM,TN=4, BK=16`}	3.0861

Table 19: Top 10 configurations for the 2D Register Tiling implementation, ranked by execution time. The parameter space evaluated is: $BM, BN \in \{32, 64, 128\}$, $TM, TN \in \{4, 8\}$, and $BK \in \{8, 16, 32\}$.

As demonstrated by the results in Table 19, the optimal configuration to maximize arithmetic intensity and fully exploit the GPU's memory hierarchy is achieved using a Block Tile size of 128×128, a step size of 32, and a "Thread Tile" size of 8×8. By assigning a 8×8 sub-matrix to each thread, we ensure massive data reuse directly within the fast registers. Then, the large 128×128 block tile mitigates the latency of Global Memory accesses, while a step size of $BK = 32$ allows a perfect balance: it guaranties a perfect coalesced 128-byte memory transactions during the loading phase into Shared Memory. Since a single-precision float occupies 4 bytes and a CUDA Warp consists of exactly 32 threads, a contiguous memory request perfectly aligns with the hardware's optimal transaction size ($4 \times 32 = 128$ bytes), allowing to avoid Shared Memory overflow and prevents register spilling.

5.4 Results & Comparison

Starting from the crucial role of the arithmetic precision, we investigated the difference in performance between single-precision (FP32) and double-precision (FP64) format. The

benchmark was executed using a fixed matrix size of 15000×15000, and the results are illustrated in Figure 15.

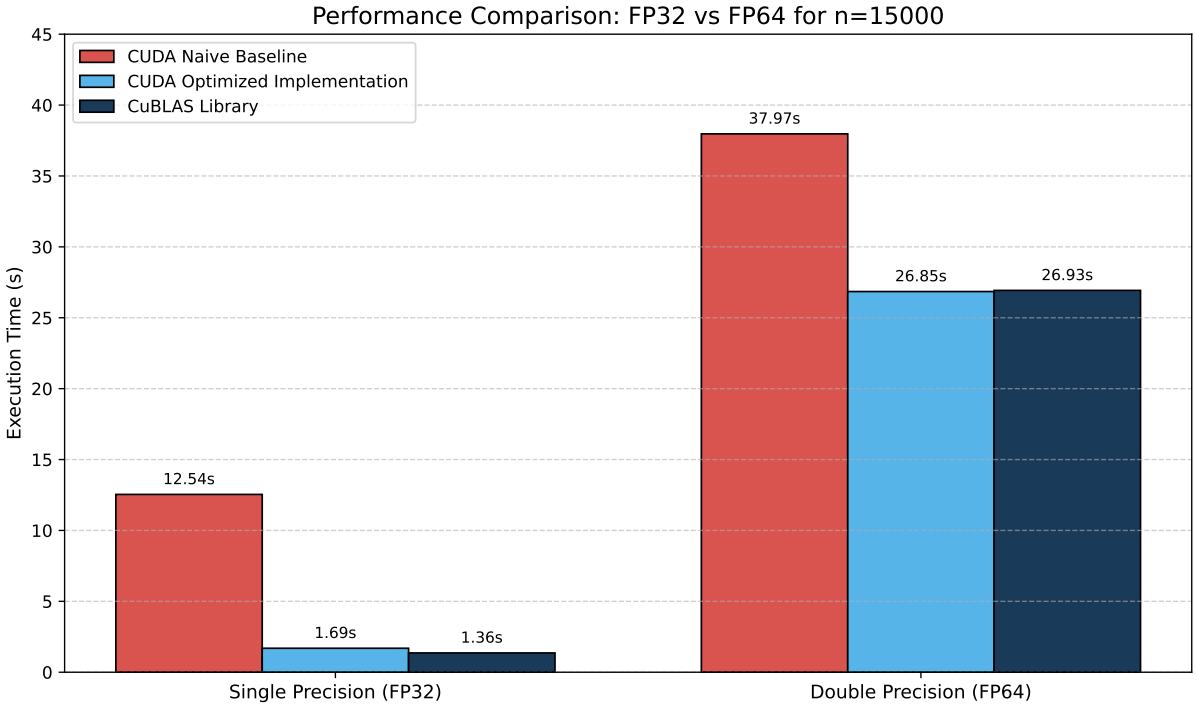

Figure 15: Comparative performance analysis of the single-precision (FP32) and double-precision (FP64) executions over a matrix size with fixed $n = 15000$.

Figure 15 denotes how utilizing the single-precision (FP32) format instead of the double-precision (FP64), it delivers a substantial improvement in terms of performance across all implementations, reducing execution times by approximately 25 seconds. The motivations remain consistent with those already detailed in Section 5.1.1: FP32 allows to fully exploit the GPU's computational resources, as the Tesla T4 is equipped with a significant higher number of FP32 cores compared to FP64.
Subsequently, Table 20 and Figure 16 demonstrate the effectiveness of the implemented optimization strategies, inspired by the cuBLAS library. The performance gap of our custom kernel compared to the naive implementation is significant, successfully overcoming its inherent architectural bottlenecks, while the difference with the cuBLAS reference library is minimal, successfully achieving the desired computational performance.

Matrix Size (n)	**Naive (s)**	**Optimized (s)**	**cuBLAS (s)**
10000	3.27	0.53	0.44
15000	12.54	1.69	1.36
20000	30.32	3.97	3.51

Table 20: Execution times for single-precision (FP32) matrix multiplication comparing the naive, optimized, and cuBLAS implementations.

Finally, we report the results obtained through the application of memory vectorization using the `float4` data type. As summarized in Table 21, this technique confirms further

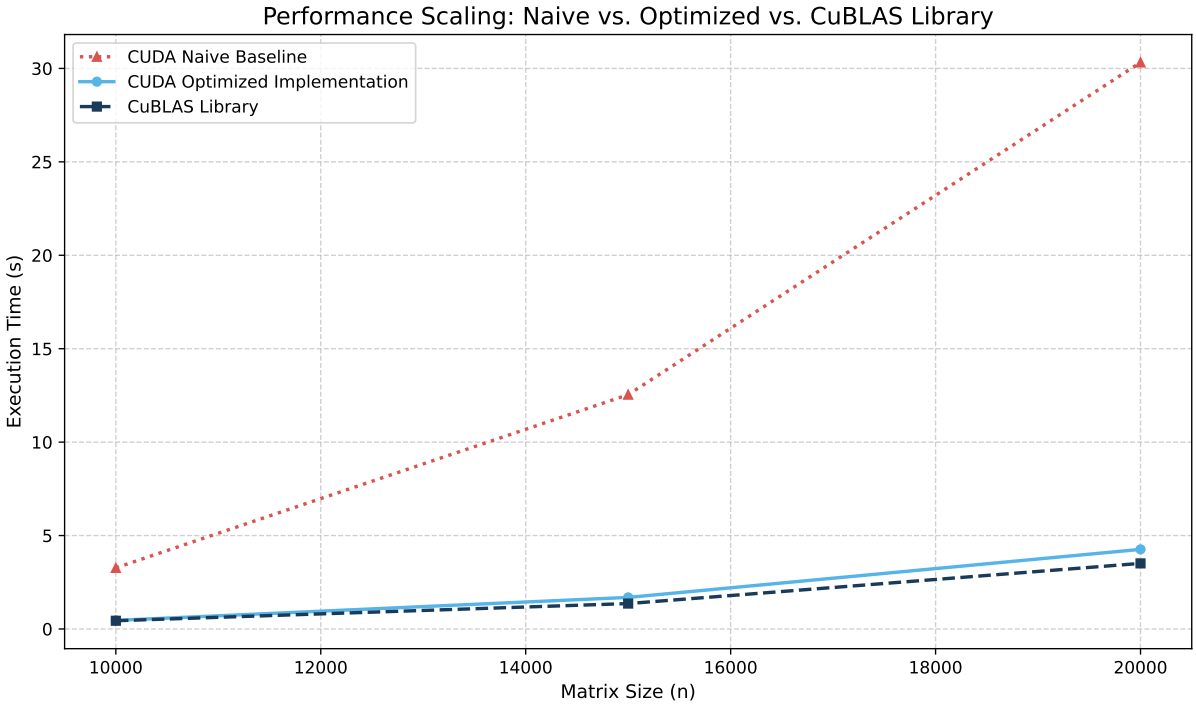

Figure 16: Execution time comparison among the naive baseline, the optimized version, and the cuBLAS library across varying matrix sizes.

improvements upon our optimized version and the successful reduction of the performance gap with the cuBLAS benchmark to a negligible discrepancy.

Matrix Size (n)	**float4 (s)**	**cuBLAS (s)**
10000	0.45	0.44
15000	1.54	1.36
20000	3.70	3.51

Table 21: Comparison of the execution times between the memory-vectorized (`float4`) implementation and the proprietary cuBLAS reference library.

6 Conclusions & Results

This final section provides extensive heuristics and a comprehensive cross-paradigm comparison of all developed implementation to assess their relative efficiency and scalability. Using the Sequential Naive version as a universal baseline, we analyze the impact of sequential optimizations, multi-threading, distributed computing, and GPU acceleration with a fixed matrix size of $n = 10000$. The following analysis highlights the performance evolution from the naive sequential code to our custom-optimized versions, ultimately comparing them against industry-standard library references to evaluate the achieved hardware utilization.

6.1 Sequential

6.1.1 Experimental Data and Speedup

- **Execution Times:** $T_{naive} = 76.98$ s, $T_{opt} = 44.37$ s, $T_{cblas} = 32.73$ s.
- **Speedup ($S = T_{naive}/T_{target}$):** Since $N_{threads} = 1$, Efficiency $E = S$.
  - $S_{opt} = 76.98/44.37 \approx 1.74$
  - $S_{cblas} = 76.98/32.73 \approx 2.35$

6.1.2 Throughput and Roofline Metrics

For a square matrix multiplication of size $N \times N$, the total computational cost is approximated as $2N^3$ FLOP, since each of the N^2 output elements requires about N multiplications and N additions. For $N = 10000$, this gives $Ops = 2 \cdot (10000)^3 = 2 \cdot 10^{12}$ FLOP. Therefore, the throughput is

$$GFLOPS = \frac{Ops}{T \cdot 10^9} = \frac{2 \cdot 10^{12}}{T \cdot 10^9} = \frac{2000}{T}.$$

Version	GFLOPS	L1 AI (FLOP/Byte)
Naive	25.98	0.20
Optimized (Tiling)	45.07	0.20
Library (CBLAS)	61.11	0.75

Table 22: Sequential metrics comparison for $N = 10000$

6.2 OpenMP

6.2.1 Execution Times and Scheduling Policies

The performance of parallel implementations, depicted in Table 23, depends on the workload distribution across 24 threads.
In the remainder of this section, the Naive version is considered with *guided* scheduling according to Table 23, the Optimized version with *dynamic* scheduling and chunk size 4 according to Table 12, while the Library version is treated separately since its performance is scheduling-independent, as shown again in Table 23.

Implementation	Static (s)	Dynamic (s)	Guided (s)
Naive	28.97	27.64	21.19
Optimized	7.16	6.66	6.63

Implementation	Scheduling-Independent Performance (s)
Library	4.19

Table 23: Execution times of the parallel implementations under different OpenMP scheduling policies. The library-based version is reported separately because its performance is independent of the scheduling strategy adopted for the loop-based implementations.

6.2.2 Parallel Metrics (Speedup and Efficiency)

Table 24 reports the Speedup (S) and Efficiency (E) of the three parallel versions against the sequential naive execution time, i.e. $T_{seq_naive} = 76.98$ s.

Version	Speedup (S)	Efficiency (E)
Naive	3.63	15.1%
Optimized	11.61	48.4%
Library	18.37	76.5%

Table 24: Parallel performance metrics.

6.2.3 Parallel Throughput and Roofline Metrics

Version	GFLOPS	L1 AI (FLOP/Byte)
Naive	94.38	0.105
Optimized	301.66	0.200
Library	477.33	0.750

Table 25: Parallel Roofline metrics ($P = 24$)

6.3 MPI

6.3.1 Experimental Data

The execution times seen in table 26 measured on a distributed memory system with $N = 10000$. Speedup (S) is relative to $T_{seq_naive} = 76.98$ s.

6.3.2 Distributed Speedup, Efficiency, and Throughput

To better quantify the benefits of the distributed implementations, Table 27 reports the Speedup (S), Efficiency (E), and achieved throughput in GFLOPS for the Naive, Optimized, and Library versions.

Implementation	Processes (P)	Time (s)
Naive	25	10.48
Optimized (SUMMA)	25	7.71
Library (ScaLAPACK)	16	6.33

Table 26: MPI execution times for $N = 10000$. Each version is measured with its fastest measured number of processes, so a oversubscribed SUMMA is faced against a 16 thread ScaLAPACK.

Version	Speedup	Efficiency (%)	GFLOPS
Naive	7.35	29.4	190.84
Optimized	9.98	39.9	259.40
Library	12.16	76.0	315.96

Table 27: Distributed performance metrics

6.4 CUDA

6.4.1 Theoretical Peak and Roofline Parameters

The NVIDIA Tesla T4 provides a theoretical peak of 8.1 TFLOPS (FP32) with a memory bandwidth of 320 GB/s[2].

$$Ridge_Point = \frac{8100 \text{ GFLOPS}}{320 \text{ GB/s}} \approx 25.3 \text{ FLOP/Byte}$$

Given the high arithmetic intensity of matrix multiplication, the problem is theoretically Compute-Bound.

6.4.2 Experimental Results and Throughput

To summarize the performance of the GPU implementations, Table 29 reports the achieved Speedup, Efficiency, and throughput in GFLOPS for the Naive, Optimized, and cuBLAS-based versions. As in the previous subsections, speedup (S) is computed relatively to $T_{seq_naive} = 76.98$ s.

6.4.3 Experimental Results and Throughput

To provide a clearer view of the GPU results, Table 28 reports the measured execution times, while Table 29 summarizes the corresponding Speedup, Efficiency, and achieved throughput in GFLOPS for the Naive, Optimized, and cuBLAS-based implementations. Unlike the CPU-based sections, where efficiency is defined as $E = S/p$, in the CUDA setting efficiency is reported as the ratio between the achieved throughput and the theoretical FP32 peak of the Tesla T4, namely

$$E_{gpu} = \frac{\text{GFLOPS}_{achieved}}{\text{GFLOPS}_{theoretical\ peak}} \times 100 = \frac{\text{GFLOPS}_{achieved}}{8100} \times 100.$$

Version	Time (s)
Naive	3.27
Optimized (Shared Memory)	0.46
Library (cuBLAS)	0.44

Table 28: GPU execution times

Version	Speedup	Efficiency (%)	GFLOPS
Naive	23.54	7.5	611.62
Optimized (Shared Memory)	167.35	53.7	4347.83
Library (cuBLAS)	174.95	56.1	4545.45

Table 29: GPU performance metrics

6.5 Final Comparison

The table below compares all tested implementations.
The Speedup (S) is calculated relative to the **Sequential Naive** baseline ($T = 76.98$ s, 25.98 GFLOPS).

6.5.1 Global Performance Comparison ($N = 10000$)

Paradigm	Implementation	Time (s)	GFLOPS	Speedup
Sequential	**Naive**	**76.98**	**25.98**	**1.00x**
	Optimized	44.37	45.07	1.73x
	Library (CBLAS)	32.73	61.11	2.35x
OpenMP	Naive	21.19	94.38	3.63x
	Optimized	6.63	301.66	11.61x
	Library	4.19	477.33	18.37x
MPI	Naive ($P = 25$)	10.48	190.84	7.35x
	Optimized ($P = 25$)	7.71	259.40	9.98x
	Library ($P = 16$)	6.33	315.96	12.16x
CUDA	Naive	3.27	611.62	23.54x
	Optimized	0.46	4347.83	167.35x
	Library (cuBLAS)	0.44	4545.45	174.95x

Table 30: Full comparison of Matrix Multiplication implementations. The **Sequential Naive** version serves as the universal baseline for speedup calculations.

6.5.2 Conclusions

From the final comparison, a clear hierarchy emerges among the optimization and parallelization paradigms. Sequential optimization alone already achieves a meaningful improvement over the naive baseline, proving that memory locality and compiler optimizations are fundamental even in the absence of explicit parallelism. OpenMP offers the

best performances on a single multicore workstation, combining relatively low programming complexity with large speedups, although its scalability is ultimately constrained by thread-management overhead, load imbalance, and the non homogeneous architecture of the CPU. MPI extends the computation into the distributed-memory setting, revealing how increasing performances in such an environment requires attention to communication overhead and communication topology. Finally, CUDA stands out as the dominant solution for this workload: once the computation is restructured around the GPU memory hierarchy, the achieved throughput is at least one order of magnitude higher than the CPU-based alternatives. Another important conclusion is that optimized libraries such as CBLAS, ScaLAPACK, and cuBLAS remain the reference implementation throughout the study, because they exploit advanced low-level optimizations that are difficult to reproduce on high level programming languages like C or Python. Therefore, the project confirms that performance portability is limited: each architecture requires a different optimization strategy, and the highest efficiency is obtained only when the code is explicitly tailored to that specific execution environment.

References

[1] NVIDIA Corporation. *cuBLAS Library User Guide*, 2024. Release: 12.x.

[2] NVIDIA Corporation. NVIDIA T4 Tensor Core GPU for AI Inference. `https://www.nvidia.com/en-us/data-center/tesla-t4/`, 2026. Accessed: 2026-04-14.

[3] Robert A. Van De Geijn and Jerrell Watts. SUMMA: Scalable universal matrix multiplication algorithm. Technical Report LAPACK Working Note 96, University of Texas at Austin, 1995.